

Army — A Better Way to Write SQL in Java, A Better Blocking ORM Framework

Contents

1. Preface	7
1.1. Introduction	7
1.2. Design Philosophy	7
1.3. Project Conventions	7
1.4. Core Concepts at a Glance	7
2. Quick Start	8
2.1. Maven Dependency Configuration	8
2.1.1. Core Dependency	8
2.1.2. Annotation Processor Configuration	8
2.2. Your First Domain Class	8
2.3. Compile-Time Metamodel Generation	9
2.4. Creating a SessionFactory	10
2.4.1. Blocking Mode	10
2.4.2. What's Required vs. Optional	11
2.5. Your First Insert	12
2.6. Your First Query	13
3. Domain Model Definition	13
3.1. Annotation Overview	13
3.2. @Table — Table Mapping	14
3.2.1. Attributes	15
3.2.2. Examples from stock-data	15
3.3. @Column — Column Mapping	16
3.3.1. Built-in Conventions (Zero-Config)	16
3.3.2. Reserved Fields	17
3.3.3. Attributes	17
3.3.4. Override via TableMeta.properties	18
3.3.5. Examples from stock-data	19
3.4. @Generator / @GeneratorType — Field Value Generation	19
3.4.1. Generator Types	19
3.4.2. Override via TableMeta.properties	20
3.4.3. Attributes	20
3.4.4. Examples from stock-data	20

3.5. @Param / @FieldParam / @OverrideParams — Generator Parameters	21
3.6. @Mapping — Custom Type Mapping	22
3.6.1. Attributes	22
3.6.2. Examples from stock-data	22
3.7. @Index — Index Definitions	23
3.7.1. Attributes	23
3.7.2. Index Name Convention	23
3.7.3. Examples from stock-data	23
3.7.4. @IndexField — Per-Column Index Configuration	24
3.8. @MappedSuperclass — The Base Domain Pattern	24
3.8.1. The Inheritance Hierarchy	25
3.8.2. Code Example	25
3.8.3. Key Points	26
3.9. @Inheritance / @DiscriminatorValue — Single-Table Inheritance	26
3.10. @DdlMode — DDL Generation Control	27
3.11. @Codec — Reserved for Future Use	27
3.12. TableMeta.properties — Runtime Configuration	28
3.12.1. Placeholder Expressions Reference	28
3.12.2. What Happens When Not Configured	28
3.12.3. Usage Examples	31
3.12.4. Property Key Format	32
3.12.5. Complete Configuration Example	33
4. MappingType — Java-to-Database Type Mapping	34
4.1. Overview: MappingType Interface	34
4.2. Default Type Inference	35
4.2.1. Built-in Java-to-MappingType Lookup	35
4.2.2. Enum Type Inference	36
NameEnumType — Default Enum Strategy	36
TextEnumType — Text-Based Enum with Custom Label	37
CodeEnumType — Integer-Code Enum	38
PostgreSQL Native ENUM via @DefinedType	39
Special Enums: Month and DayOfWeek	39
4.3. Navigation: stock-data Example	40
4.4. @Mapping Annotation: Explicit Mapping Control	40
4.4.1. Annotation Definition	40
4.4.2. Resolution Priority	41
4.4.3. Example: Mapping a String to TEXT (Long Content)	41
4.4.4. Example: Mapping a String to CHAR (Fixed-Length)	42
4.4.5. Example: Runtime-Configurable Mapping	42
4.5. Array Types	42
4.5.1. Default Array Inference	42

4.5.2. Custom Array Mapping	43
4.6. Composite Type (PostgreSQL Row Types)	43
4.7. Vector Type (pgvector Embeddings)	45
4.7.1. Vector Store Example (army-spring-ai-vector-store)	45
4.7.2. Vector Store Domain Subclass Example	46
4.8. JSON / JSONB Types	46
4.8.1. Metadata Storage (army-spring-ai-vector-store)	47
4.8.2. Specialized Data Storage (army-spring-ai-model-chat-memory)	47
4.8.3. List Storage in JSONB (UploadRecord)	48
4.9. Text vs Varchar — Choosing the Right String Mapping	48
4.10. How TypeMappingHandler Resolves to SQL Types	48
4.11. Custom MappingType — Extending the Type System	50
4.11.1. Step 1: Choose the Right Base Class	50
4.11.2. Step 2: Implement the Corresponding <code>Sql*</code> Marker Interface	50
4.11.3. Step 3: Follow the Implementation Pattern	51
4.11.4. Array Type Example	53
4.11.5. Quick Checklist	55
4.12. Full Mapping Chain Summary	56
5. Criteria API — Writing SQL	56
5.1. Standard Criteria API	56
5.1.1. The SQLs Utility Class	57
5.1.2. SELECT Statements	57
Basic SELECT	57
Full Clause Chain	58
5.1.3. INSERT Statements	58
Single Table INSERT	58
Child INSERT (Parent-Child Hierarchy)	66
INSERT Options	67
5.1.4. UPDATE Statements	67
Single Table UPDATE	67
Domain UPDATE	67
Batch UPDATE	68
5.1.5. DELETE Statements	69
Single Table DELETE	69
Domain DELETE	69
5.1.6. Standard Functions	70
Aggregate Functions	70
Scalar Functions	71
5.1.7. Standard Window Functions	71
5.1.8. Conditional Expressions	73
5.1.9. Logical Operators	73

5.1.10. Default Parameter Binding	73
The default path: <code>field.equal(value)</code>	73
What happens with different value types	74
Right operand is already an <code>Expression</code>	74
Left operand has no type information	75
Right operand could trigger an error	75
Controlling the binding mode: <code>LiteralMode</code>	75
Explicit binding with <code>BiFunction</code> overloads	77
5.2. CTE (Common Table Expressions)	78
5.3. Subqueries	78
5.3.1. Scalar Subquery	78
5.3.2. Table Subquery	79
5.4. MySQL Dialect API	79
5.4.1. The MySQLs Utility Class	79
5.4.2. MySQL-Specific Features	79
5.5. PostgreSQL Dialect API	80
5.5.1. The Postgres Utility Class	80
5.5.2. PostgreSQL-Specific Features	80
6. Session API — Executing SQL	81
6.1. SessionFactory Configuration	81
6.1.1. Configuration Options	81
6.2. Session Operations	81
6.2.1. Obtaining a Session	81
Local Session (recommended for short-lived operations)	81
Regular Session	81
6.2.2. Query Operations	82
6.2.3. Update Operations	82
6.2.4. Batch Operations	82
6.3. Transaction Management	82
6.3.1. Blocking-Mode Transactions	82
6.3.2. Savepoints	83
6.3.3. Transaction Isolation Levels	83
7. Core Architecture	83
7.1. Module Breakdown	83
7.1.1. <code>army-core</code> (Core Module)	83
7.1.2. <code>army-annotation</code> (Annotation Module)	83
7.1.3. <code>army-mysql</code> (MySQL Dialect Module)	83
7.1.4. <code>army-postgre</code> (PostgreSQL Dialect Module)	83
7.1.5. <code>army-oracle</code> (Oracle Dialect Module)	84
7.1.6. <code>army-sqlite</code> (SQLite Dialect Module)	84
7.1.7. <code>army-jdbc</code> (JDBC Executor Module)	84

7.1.8. army-spring (Spring Integration Module)	84
7.1.9. army-sync (Blocking Mode Module)	84
7.1.10. army-struct (Foundation Structures Module)	84
7.1.11. army-spring-ai-model-chat-memory / army-spring-ai-vector-store	84
7.2. Architectural Layers	84
7.2.1. Presentation Layer (Criteria API)	84
7.2.2. Parsing Layer (Dialect Parser)	85
7.2.3. Metadata Layer (Meta)	85
7.2.4. Execution Layer (Executor)	85
7.2.5. Session Layer (Session)	85
7.2.6. Type Mapping Layer (Mapping)	85
8. Type Mapping System	85
8.1. Built-in Type Mappings	85
8.1.1. Numeric Types	85
8.1.2. String Types	85
8.1.3. Date-Time Types	86
8.1.4. Other Types	86
8.2. Array Types	86
8.3. JSON Types	86
8.4. Custom Type Mappings	86
8.4.1. Implementing the MappingType Interface	86
8.4.2. Registering Custom Types	86
8.5. JSON / XML Codecs	86
9. Field Generators	87
9.1. Snowflake Generator	87
9.1.1. Configuration Parameters	87
9.2. UUID Generator	87
9.3. Custom Generators	87
9.3.1. Implementing the FieldGenerator Interface	87
9.3.2. Registering a Generator Factory	87
10. Spring Integration	88
10.1. SessionFactory Bean Configuration	88
10.1.1. Blocking Mode	88
10.2. Spring Transaction Management	88
10.2.1. Blocking Transaction Manager	88
10.3. Declarative Transactions	88
10.4. DataSource Integration	88
11. Database Dialect Support Matrix	89
11.1. MySQL	89
11.2. PostgreSQL	89
11.3. SQLite	89

11.4. Oracle	89
12. Advanced Features	89
12.1. Reserved Field Mechanism	89
12.2. Soft Deletes	89
12.3. Optimistic Locking	90
12.4. Inheritance Mapping in Depth	90
12.5. Parameter Binding Modes	90
12.6. SQL Logging	90
12.7. Batch Operation Optimization	91
12.8. Result Set Mapping (RowMaps)	91
13. Schema Comparison & Synchronization	91
13.1. SchemaComparer	91
13.2. Comparison Process	91
13.3. Supported Comparison Items	91
13.4. PostgreSQL Composite Type Synchronization	91
14. Best Practices	91
14.1. Domain Class Design	91
14.2. Criteria API Usage	92
14.3. Transaction Management	92
14.4. Performance Optimization	92
14.5. Type Mapping	92
15. FAQ	92
15.1. Metamodel Classes Not Generated by the Annotation Processor	92
15.2. Snowflake Generator Duplicate startTime Warning	93
15.3. Transaction Exception (IllegalTransactionStateException)	93
15.4. Type Mapping Errors	93
16. Appendix	93
16.1. Reserved Fields Reference	93
16.2. Database Support Matrix	93
16.3. Version History	94
16.3.1. 0.6.8-SNAPSHOT (Current)	94
16.3.2. 0.6.6	94
16.3.3. 0.6.0	94
16.4. Related Resources	94
16.5. License	94

© 2023-present The original authors.



Copies of this document may be made for your own use and for distribution to others, provided that you do not charge any fee for such copies and further provided that each copy contains this Copyright Notice, whether distributed in

1. Preface

1.1. Introduction

Army is a better Java SQL framework offering the following core capabilities:

- A better way to write SQL in Java
- A better blocking ORM framework
- A higher-level database driver

1.2. Design Philosophy

1. **Don't create a new world — just map the real world.** Respect database standards and dialect differences.
2. **We need standards, and we need dialects.** That's how the real world works.
3. **Keep it simple — blocking by default, straightforward and predictable.**

1.3. Project Conventions

- Interfaces or classes whose names begin with an underscore (`_`) are **private to the Army framework** and should not be directly depended upon in user code.

1.4. Core Concepts at a Glance

Using Army follows a clear pipeline:

Domain classes (annotated)

- Annotation processor generates metamodel classes at compile time
- Criteria API builds SQL statements
- Dialect parser generates native SQL
- Session API executes statements
- Executor (`StmntExecutor`) drives JDBC execution

- **Domain Class** — A POJO annotated with `@Table`, `@Column`, etc., describing the mapping between Java objects and database tables.
- **Metamodel Class** — A `_`-suffixed class auto-generated at compile time by `ArmyMetaModelDomainProcessor`, providing type-safe field references.
- **Criteria API** — A Java DSL for building SQL statements (SELECT, INSERT, UPDATE, DELETE).
- **Session API** — A unified entry point for executing statements and managing transactions.
- **Dialect Parser** — Translates standard Criteria into native SQL for each database.

- **Executor** — `io.army.executor.StmtExecutor`, responsible for JDBC statement execution, parameter binding, and result set processing.

2. Quick Start

2.1. Maven Dependency Configuration

2.1.1. Core Dependency

```
<dependencies>
  <dependency>
    <groupId>io.qinarmy</groupId>
    <artifactId>army-jdbc</artifactId>
    <version>0.6.6</version>
  </dependency>
  <dependency>
    <groupId>io.qinarmy</groupId>
    <artifactId>army-postgre</artifactId>
    <version>0.6.6</version>
  </dependency>
</dependencies>
```

2.1.2. Annotation Processor Configuration

Configure the annotation processor in the Maven module that contains your domain classes:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>

<annotationProcessors>io.army.modelgen.ArmyMetaModelDomainProcessor</annotationProcess
ors>

      </configuration>
    </plugin>
  </plugins>
</build>
```

2.2. Your First Domain Class

Map a Java class to a database table using annotations:

```
@Table(name = "stock",
```

```

        indexes = @Index(name = "uni_stock_exchange_code", fieldList = {"exchange",
"code"}, unique = true),
        comment = "stock")
public class Stock {

    //auto-increment primary key use below:
    // @Generator(type = GeneratorType.POST)
    @Generator(value = "io.army.generator.snowflake.Snowflake8Generator",
        params = @Param(name = "startTime", value = "1779012232202"))
    @Column
    public long id;

    @Column
    public LocalDateTime createTime;

    @Column
    public LocalDateTime updateTime;

    @Column(comment = "version")
    public int version;

    @Column(notNull = true, updatable = false, precision = 5, comment = "exchange
code")
    public String exchange;

    @Column(notNull = true, updatable = false, precision = 15, comment = "stock code")
    public String code;

    @Column(notNull = true, precision = 130, comment = "company name")
    public String name;

    @Column(notNull = true, precision = 10, defaultValue = "'NORMAL'", comment =
"listing status")
    public StockStatus status;

    @Column(precision = 10, scale = 2, defaultValue = "0.00", comment = "offer price")
    public BigDecimal offerPrice;

    @Column(defaultValue = "DATE '1970-01-01'", comment = "listing date")
    public LocalDate listingDate;

}

```

2.3. Compile-Time Metamodel Generation

`ArmyMetaModelDomainProcessor` automatically processes `@Table`-annotated classes at compile time and generates a static metamodel class `Stock_`:

```

@Generated(value = "io.army.modelgen.ArmyMetaModelDomainProcessor",

```

```

        date = "2024-01-02 07:13:54.605524+08:00",
        comments = "stock")
public abstract class Stock_ {

    private Stock_(){
        throw new UnsupportedOperationException();
    }

    public static final SimpleTableMeta<Stock> T;

    static {
        T = _TableMetaFactory.getSimpleTableMeta(Stock.class);

        final int fieldSize = T.fieldList().size();
        if(fieldSize != 10){
            throw _TableMetaFactory.tableFiledSizeError(Stock.class,fieldSize);
        }
    }

    // Field name constants (type-safe references)
    public static final String NAME = "name";
    public static final String OFFER_PRICE = "offerPrice";
    public static final String EXCHANGE = "exchange";
    public static final String CODE = "code";
    // ... other fields

    // Field metadata
    public static final FieldMeta<Stock> name = T.field(NAME);
    public static final FieldMeta<Stock> offerPrice = T.field(OFFER_PRICE);
    public static final PrimaryFieldMeta<Stock> id = T.id();
    // ... other field metadata
}

```



Metamodel classes are the key to building **type-safe** Criteria statements. All field references go through metamodel constants, eliminating string-based typos.

2.4. Creating a SessionFactory

2.4.1. Blocking Mode

```

public void createSessionFactory() {
    final Map<String, Object> envMap = new HashMap<>();
    envMap.put(ArmyKey.DATABASE.name, Database.MySQL);
    envMap.put(ArmyKey.DIALECT.name, MySQLDialect.MySQL80);

    final DataSource dataSource = createDataSource();
    sessionFactory = SyncFactoryBuilder.builder()
        .name("army-test")

```

```

        .packagesToScan(List.of("io.army.example.stock.domain"))
        .datasource(dataSource)
        .environment(StandardEnvironment.from(envMap))
        .build();
    }

```

2.4.2. What's Required vs. Optional

FactoryBuilder methods come with clear markers in the source code. Here's the breakdown:

Method	Status	Notes
<code>name(String)</code>	Required	SessionFactory name, must be non-empty.
<code>datasource(Object)</code>	Required	JDBC <code>DataSource</code> instance. Can be a <code>ReadWriteSplittingDataSource</code> for read/write splitting.
<code>packagesToScan(List)</code>	Required	List of packages where domain classes (annotated with <code>@Table</code>) reside.
<code>environment(ArmyEnvironment)</code>	Required	Environment configuration passed via <code>StandardEnvironment.from(map)</code> . Must contain at least <code>DATABASE</code> and <code>DIALECT</code> (see below).
<code>classLoader(ClassLoader)</code>	Optional	Default: <code>Thread.currentThread().getContextClassLoader()</code>
<code>loadStaticModel(boolean)</code>	Optional	Default <code>false</code> . Enable to load static metamodel classes after <code>TableMeta</code> creation for extra safety.
<code>validateOnStartup(boolean)</code>	Optional	Default <code>false</code> . Enable for stronger consistency validation at startup.
<code>jsonCodec(JsonCodec)</code> / <code>xmlCodec(XmlCodec)</code>	Optional	Custom JSON/XML codec.
<code>fieldGeneratorFactory(FieldGeneratorFactory)</code>	Optional	Custom field generator factory.

ArmyKey environment entries — only two are truly mandatory, all others have sensible defaults:

ArmyKey	Required?	Notes
<code>DATABASE</code>	Yes	<code>Database.MySQL</code> / <code>Database.PostgreSQL</code> / <code>Database.SQLite</code> . No default — framework needs to know which database to operate against.

ArmyKey	Required?	Notes
DIALECT	Yes	MySQLDialect.MySQL80 / PostgreDialect.POSTGRE15 / SQLiteDialect.SQLite34. No default — must match your database version.
ZONE_OFFSET	No	Timezone offset. When omitted, the framework uses database defaults.
DDL_MODE	No	Default: DdlMode.UPDATE (create tables if absent, update columns if changed).
VISIBLE_MODE	No	Default: AllowMode.NEVER (disable soft-delete visible field).
SQL_LOG_MODE	No	Default: SqlLogMode.OFF. Set to SqlLogMode.BEAUTIFY_DEBUG during development.
All other keys	No	Default values are defined in ArmyKey / SyncKey. See the source for the complete list.

That means the minimal environment setup looks like the example above — just **DATABASE** and **DIALECT**. In the test example at **FactoryUtils.java**, extra keys like **SQL_LOG_MODE** and **VISIBLE_MODE** are set for development convenience, but they are not required for production.



If you forget **DATABASE** or **DIALECT**, the framework throws **IllegalStateException** with a clear message — no silent failures.

2.5. Your First Insert

```
// 1. Build the insert statement
final Insert stmt;
stmt = SQLs.singleInsert()
    .insertInto(Stock_.T)
    .values(stockList) // List<Stock> – id & managed fields are auto-generated
    .asInsert();

// 2. Execute the insert
try (SyncLocalSession session = sessionFactory.localSession()) {
    long rows;
    rows = session.update(stmt);
    System.out.println("Inserted " + rows + " row(s)");
}
```

Generated SQL (PostgreSQL)

```
INSERT INTO stock (id, create_time, update_time, version, exchange,
                  code, name, status, offer_price, listing_date)
VALUES (?, ?, ?, ?, ?,
        ?, ?, ?, ?, ?)
, ( ?, ?, ?, ?, ?,
    ?, ?, ?, ?, ?) -- one row per Stock object
```

, ...

2.6. Your First Query

```
// 1. Build the query statement
final Select stmt;
    stmt = SQLs.query()
        .select(Stock_.id, Stock_.name, Stock_.offerPrice)
        .from(Stock_.T, AS, "s")
        .where(Stock_.name.equal("Tesla"))
        .asQuery();

// 2. Execute the query
try (SyncLocalSession session = sessionFactory.localSession()) {
    Stock stock ;
    stock = session.queryOne(stmt, Stock.class);
    System.out.println(stock);
}
```

Generated SQL (PostgreSQL)

```
SELECT s.id, s.name, s.offer_price
FROM stock AS s
WHERE s.name = ? -- param: 'Tesla'
```

3. Domain Model Definition

Domain model definition is the starting point of Army. This chapter walks you through how to use annotations to tell the framework: *"this Java class maps to that database table, this field maps to that column, this field needs a Snowflake ID generator..."* — all in plain Java, no XML.

Behind the scenes, Army's `TableMetaUtils` and `FieldMetaUtils` read these annotations at runtime to build `TableMeta` objects, which are the immutable metadata blueprints that drive SQL generation. The entry point is `DefaultTableMeta#getTableMeta(Class, MetaContext)`, which reads `@Table`, walks the class hierarchy, creates `FieldMeta` instances via `TableFieldMeta#createFieldMeta`, and caches the result — so each domain class is parsed only once.

3.1. Annotation Overview

Here's a quick reference of all Army annotations. Don't worry about memorizing them now — each is explained in detail with real examples below.

Annotation	Purpose	Target
<code>@Table</code>	Maps a class to a database table (name, comment, indexes)	Class

Annotation	Purpose	Target
<code>@Column</code>	Maps a field to a database column (name, type, nullability, default)	Field
<code>@Generator</code>	Specifies how a field's value is generated (e.g., Snowflake ID)	Field
<code>@GeneratorType</code>	When generation happens: <code>PRECEDE</code> / <code>POST</code> / <code>RUNTIME</code> / <code>DEFAULT</code>	Enum
<code>@Param</code>	A name-value pair passed to a <code>FieldGenerator</code>	Annotation member
<code>@FieldParam</code>	Binds a field name to parameter overrides (inside <code>@OverrideParams</code>)	Annotation member
<code>@OverrideParams</code>	Overrides generator parameters of inherited fields	Class
<code>@Index</code>	Defines a database index (name, fields, uniqueness, type)	Class (inside <code>@Table</code>)
<code>@IndexField</code>	Per-column index configuration (collation, opclass, sort)	Class (inside <code>@Index</code>)
<code>@SortOrder</code>	Index column sort: <code>ASC</code> / <code>DESC</code> / <code>DEFAULT</code>	Enum
<code>@NullsOrder</code>	Index column NULL ordering: <code>NULLS_FIRST</code> / <code>NULLS_LAST</code> / <code>DEFAULT</code>	Enum
<code>@Mapping</code>	Custom Java↔SQL type mapping	Field
<code>@Inheritance</code>	Marks the root of a single-table inheritance hierarchy	Class
<code>@DiscriminatorValue</code>	Discriminator enum value for a child class in inheritance	Class
<code>@MappedSuperclass</code>	Marks a base class whose fields are inherited by subclasses	Class
<code>@DdlMode</code>	Controls DDL generation: <code>CREATE</code> / <code>NONE</code> / <code>DEFAULT</code>	Enum
<code>@Codec</code>	Marker for future custom encoding/decoding	Field



Many annotation attributes support **placeholders** (`${DEFAULT}`, `${RUNTIME}`, `${OPTIONAL}`, `${DEFAULT_VALUE}`) that are resolved at runtime from the configuration file `META-INF/army/TableMeta.properties` on the classpath. See [TableMeta.properties — Runtime Configuration](#) for details.

3.2. @Table — Table Mapping

`@Table` is placed on a Java class to tell Army: *"this class maps to a database table."* It controls the table name, index definitions, DDL behavior, and more. At runtime, `TableMetaUtils.tableName` and `TableMetaUtils.tableComment` resolve its values; `TableMetaUtils.schemaMeta` handles `catalog/schema`, and `TableMetaUtils.tableCreateDdl` determines whether DDL is generated.



Both `name` and `comment` are **required** attributes. If you omit them (or use empty

strings), the framework throws `MetaException` at startup. This is intentional — Army refuses to guess silently.

3.2.1. Attributes

- `name` (required) — The database table name. Must not be in camelCase. Supports placeholders: `${DEFAULT}` auto-converts the class simple name to snake_case (e.g., `StockQuotes` → `"stock_quotes"`); `${RUNTIME}` requires a property entry.
- `comment` (required) — Table comment for DDL. Supports `${DEFAULT}` (auto-derive from class simple name) and `${RUNTIME}`.
- `catalog` (optional) — Catalog name, defaults to the database default. Property key: `{className}.Table.catalog`.
- `schema` (optional) — Schema name, defaults to the user's default schema. Property key: `{className}.Table.schema`.
- `indexes` (optional) — Array of `@Index` definitions. If no index covers the `id` field, a unique primary key index is **created automatically**.
- `immutable` (optional, default `false`) — When `true`, all `UPDATE` DML on this table is disallowed. Parent tables with `@Inheritance` cannot be marked as immutable — Army detects this at startup and throws `MetaException`.
- `allColumnNotNull` (optional, default `false`) — When `true`, every column gets `NOT NULL` in DDL. Note: reserved fields (`id`, `createTime`, `updateTime`, `version`, `visible`), discriminator fields, and primitive-typed fields are always `NOT NULL` regardless of this setting.
- `ddlMode` (optional, default `CREATE`) — Controls DDL generation. See `@DdlMode`.
- `tableOptions` (optional) — Raw SQL fragment appended to `CREATE TABLE`, e.g., `"ENGINE=InnoDB DEFAULT CHARSET=utf8mb4"`.
- `partitionOptions` (optional) — SQL partition clause, e.g., `"PARTITION BY RANGE (YEAR(create_time))"`.

3.2.2. Examples from stock-data

```
// Stock: table with a unique composite index
@Table(name = "stock",
        indexes = @Index(name = "${DEFAULT_VALUE}", unique = true,
                          fieldList = {"exchange", "code"}), // auto-generates:
        uni_stock_exchange_code
        comment = "stock")
public class Stock { }

// StockQuotes: table with unique index on (stockId, date)
@Table(name = "stock_quotes",
        indexes = @Index(name = "${DEFAULT_VALUE}", unique = true,
                          fieldList = {"stockId", "date"}), // auto-generates:
        uni_stock_quotes_stockId_date
        comment = "daily stock quotes")
```

```
public class StockQuotes { }

// UploadRecord: table with multiple non-unique indexes
@Table(name = "upload_record",
    indexes = {
        @Index(name = "${DEFAULT_VALUE}", fieldList = "userId"),
        @Index(name = "${DEFAULT_VALUE}", fieldList = "fileHash")
    },
    comment = "uploaded document record")
public class UploadRecord { }
```



Use `${DEFAULT_VALUE}` as the index name to let the framework auto-generate `uni_{table}_{col}` or `idx_{table}_{col}`. Use `${DEFAULT}` only when you want to override the generated name via `META-INF/army/TableMeta.properties`. For most cases, `${DEFAULT_VALUE}` is the right choice.

3.3. @Column — Column Mapping

`@Column` maps a Java field to a database column. **Without `@Column`, a field is just a plain Java field** — not persisted, not included in any SQL, not part of the table schema. The framework's `TableMetaUtils.columnName` resolves the column name, `TableMetaUtils.columnPrecision/columnScale` resolve numeric attributes, and `FieldMetaUtils.columnComment` resolves the comment text.

3.3.1. Built-in Conventions (Zero-Config)

Army provides smart defaults for many `@Column` attributes — you only need to specify what differs from the convention:

Attribute	Convention	How It Works
<code>name</code> (not set)	<code>camelCase</code> → <code>snake_case</code>	<code>offerPrice</code> → <code>"offer_price"</code> , <code>stockId</code> → <code>"stock_id"</code>
<code>comment</code> (not set)	Auto-generated for reserved fields	<code>id</code> → <code>"primary key"</code> , <code>createTime</code> → <code>"create time"</code> , <code>updateTime</code> → <code>"update time"</code> , <code>version</code> → <code>"version that's update counter of row"</code> , <code>visible</code> → <code>"visible for soft delete"</code>
<code>precision</code> (not set)	By Java type	<code>String</code> → 255, <code>BigDecimal</code> → 16, <code>Enum</code> → max label length + 5, <code>UUID</code> → 36, <code>Vector</code> → 1024
<code>scale</code> (not set)	By Java type	<code>BigDecimal</code> → 3, temporal types (<code>LocalDateTime/OffsetDateTime/LocalTime/OffsetTime</code>) → 6 (microsecond precision)

Attribute	Convention	How It Works
<code>defaultValue</code> (not set)	By Java type	<code>□□□□</code> → <code>"0"</code> , <code>String</code> → <code>""</code> , <code>LocalTime</code> → <code>"00:00:00"</code> , <code>LocalDate</code> → <code>"1970-01-01"</code> , <code>LocalDateTime</code> → <code>"1970-01-01 00:00:00"</code>
<code>notNull</code> (not set)	<code>false</code> — except for reserved fields	<code>id/createTime/updateTime/version/visible</code> and discriminator fields are always <code>NOT NULL</code>
<code>collation</code> (not set)	Empty — no collation clause	Set explicitly only when needed, e.g., <code>"en_US.utf8"</code>



For business fields (non-reserved), if you don't provide a `comment`, Army throws `MetaException`. Reserved fields get automatic comments, but your business columns must be documented.

3.3.2. Reserved Fields

Army recognizes five field names as **reserved** — they are auto-managed by the framework and have special behavior:

Field Name	<code>NOT NULL</code>	INSERT Behavior	UPDATE Behavior
<code>id</code>	Always	Value from <code>@Generator</code>	Never updatable
<code>createTime</code>	Always	Auto-set to current time	Never updatable
<code>updateTime</code>	Always	Auto-set to current time	Always updatable , auto-refreshed on every UPDATE
<code>version</code>	Always	Auto-set to <code>0</code>	Always updatable , auto-incremented (optimistic lock)
<code>visible</code>	Always	Auto-set to <code>true</code>	Always updatable (soft delete toggle)



Reserved fields **never need** `notNull` — Army enforces it automatically. Reserved fields (except `id`) **must not have** `@Generator` — Army throws `MetaException` if you try.

3.3.3. Attributes

- `name` (optional) — Column name. If empty, `camelCase` → `snake_case`. Supports `${DEFAULT}` and `${RUNTIME}`. Property key: `{className}.{fieldName}.Column.name`.
- `comment` (optional) — Column comment for DDL. Reserved fields auto-generate. Enum fields with `${DEFAULT}` auto-generate `"enum, @see {EnumClass}"`. Business fields without a value throw `MetaException`. Supports `${DEFAULT}` and `${RUNTIME}`.
- `notNull` (optional, default `false`) — Declares `NOT NULL`. Forced `true` when: the field is reserved, a

discriminator, primitive-typed, or `Table.allColumnNotNull` is `true`.

- `insertable` (optional, default `true`) — Whether included in `INSERT`. Overridden for reserved fields: `createTime/updateTime/version/visible` are always insertable; fields with `PRECEDENCE` generators are always insertable; `id` with `POST` generator is insertable only on child tables.
- `updatable` (optional, default `true`) — Whether included in `UPDATE`. Overridden: `id/createTime/discriminator` are **never** updatable; `updateTime/version/visible` are **always** updatable; fields on `immutable` tables are never updatable.
- `precision` (optional, default `-1`) — Total digit count or length. Defaults by type (see table above). Use `Column.DEFAULT_EXP` (`-8150`) to resolve from `META-INF/Army/TableMeta.properties`.
- `scale` (optional, default `-1`) — Fractional digit count. Defaults by type (see table above). Use `Column.DEFAULT_EXP` to resolve from properties.
- `defaultValue` (optional) — SQL expression for column default. **String values must include quotes**: `"'NORMAL'"` → `DEFAULT 'NORMAL'`. Date values use SQL syntax: `"DATE '1970-01-01'"`. Supports `${DEFAULT}` for type-appropriate zero values.
- `collation` (optional) — Column collation, e.g., `"en_US.utf8"`. Property key: `{className}.{fieldName}.Column.collation`.

3.3.4. Override via TableMeta.properties

The following `@Column` attributes support placeholder expressions. Place the file at `META-INF/Army/TableMeta.properties` on the classpath:

`META-INF/Army/TableMeta.properties`

```
# File: META-INF/Army/TableMeta.properties

# Override column name
# Used when @Column(name = "${DEFAULT}") or @Column(name = "${RUNTIME}")
com.example.domain.Stock.userName.Column.name=user_name

# Override column comment
com.example.domain.Stock.status.Column.comment=custom status

# Override column precision (e.g., increase VARCHAR length to 512)
com.example.domain.Stock.name.Column.precision=512

# Override column scale (e.g., increase decimal places to 4)
com.example.domain.Stock.offerPrice.Column.scale=4

# Override column default value (SQL expression)
com.example.domain.Stock.status.Column.defaultValue='ACTIVE'

# Override column collation
com.example.domain.Stock.name.Column.collation=en_US.utf8
```



Property keys are **case-sensitive**. The path component `Column` (with capital C) is a

literal keyword — it must appear exactly as shown.

3.3.5. Examples from stock-data

```
// Simple column: auto name (exchange → exchange), NOT NULL, updatable only on INSERT
@Column(notNull = true, updatable = false, precision = 5, comment = "exchange code")
public String exchange;

// Decimal column with precision and scale → DDL: DECIMAL(10, 2)
@Column(precision = 10, scale = 2, defaultValue = "0.00", comment = "offer price")
public BigDecimal offerPrice;

// Enum column with a string default → DDL includes: DEFAULT 'NORMAL'
@Column(notNull = true, precision = 10, defaultValue = "'NORMAL'", comment = "listing
status")
public StockStatus status;

// Date column with SQL default expression
@Column(defaultValue = "DATE '1970-01-01'", comment = "listing date")
public LocalDate listingDate;

// Managed field: auto-set by the framework on INSERT, never updatable
@Column
public LocalDateTime createTime;
```

3.4. @Generator / @GeneratorType — Field Value Generation

`@Generator` controls **how a field's value is produced** — before INSERT (application-side, like Snowflake ID) or after INSERT (database auto-increment). `@GeneratorType` is the enum that tells the framework **when** to generate.

At runtime, `TableFieldMetaIs` constructor reads `@Generator` and `@GeneratorType` to create `GeneratorMeta` instances. If the type is `RUNTIME` or `DEFAULT`, it calls `FieldMetaUtils.loadGeneratorStrategy` to look up `GeneratorStrategy` from `META-INF/army/TableMeta.properties`.

3.4.1. Generator Types

Type	When to Use
<code>PRECEDE</code> (default)	Application generates the value before INSERT. Use for Snowflake IDs, UUIDs, or custom ID generation. The value from your generator is written into the <code>INSERT</code> statement.
<code>POST</code>	Database auto-increment column. No code runs — the DB assigns the value during INSERT, and Army reads it back afterward. Only supported on the <code>id</code> field , and the Java type must be <code>int</code> , <code>long</code> , <code>Integer</code> , <code>Long</code> , <code>String</code> , or <code>BigInteger</code> .

Type	When to Use
RUNTIME	Defer the decision to <code>META-INF/army/TableMeta.properties</code> . Army reads the key <code>{className}.{fieldName}.Generator.type</code> and loads a <code>GeneratorStrategy</code> . Property value format: <code>POST</code> , <code>PRECED</code> , or <code>DEFAULT</code> .
DEFAULT	Let the framework decide. For child tables in an <code>@Inheritance</code> hierarchy, defaults to <code>POST</code> ; otherwise, behaves like <code>PRECED</code> .



Discriminator fields (the field named in `@Inheritance`) and reserved fields except `id` (`createTime`, `updateTime`, `version`, `visible`) **must not** have `@Generator` — Army throws `MetaException` at startup.

3.4.2. Override via `TableMeta.properties`

When `@Generator(type = GeneratorType.RUNTIME)` is used, the actual generator strategy is loaded from `META-INF/army/TableMeta.properties`. Two keys are supported:

`META-INF/army/TableMeta.properties`

```
# File: META-INF/army/TableMeta.properties

# Option 1: Switch generator type at runtime (POST, PRECED, or DEFAULT)
com.example.domain.Stock.id.Generator.type=POST

# Option 2: Load a custom GeneratorStrategy (takes precedence over .Generator.type)
# Format: fully.qualified.StrategyClass[:jsonParams]
com.example.domain.Stock.id.GeneratorStrategy=io.army.generator.Snowflake8GeneratorStrategy:{"startTime":1779111192831}

# Override SQL type mapping for the id column (optional)
com.example.domain.Stock.id.Mapping.value=io.army.mapping.SqlBigIntType
```



The `GeneratorStrategy` value contains a colon (`:`) separating the class name from optional JSON parameters. The JSON is parsed by the strategy's static `create(String)` factory method.

3.4.3. Attributes

- `type` (optional, default `PRECED`) — The `GeneratorType` enum value.
- `value` (optional) — Fully-qualified class name of the `FieldGenerator` implementation. **Required** for `PRECED`. Ignored for `POST` (since `POST` uses the database, not application code).
- `params` (optional) — Array of `@Param` name-value pairs passed to the generator at initialization.

3.4.4. Examples from stock-data

Each entity uses the **Snowflake-8 generator** with a unique `startTime` to produce non-overlapping ID ranges across tables:

```
// Stock entity: startTime=1779012232202
@Generator(value = "io.army.generator.snowflake.Snowflake8Generator",
           params = @Param(name = "startTime", value = "1779012232202"))
@Column
public long id;

// StockQuotes entity: different startTime for a different ID space
@Generator(value = "io.army.generator.snowflake.Snowflake8Generator",
           params = @Param(name = "startTime", value = "1779111192831"))
@Column
public long id;
```



Each table should use a **different startTime** so Snowflake produces non-colliding ID ranges. This prevents primary key conflicts when joining data across tables.

3.5. @Param / @FieldParam / @OverrideParams — Generator Parameters

These three annotations work together to pass and override generator configuration. `FieldMetaUtils.getOrCreateFieldParmMap` creates the parameter map; `FieldMetaUtils.overrideParamMapIfNeed` applies `@OverrideParams` overrides.

@Param — A simple **name-value pair** passed to a generator as `@Generator(params = ...)`. The generator parses the string value into its needed type:

```
// Snowflake8Generator reads "startTime" as a long
@Generator(value = "io.army.generator.snowflake.Snowflake8Generator",
           params = @Param(name = "startTime", value = "1779012232202"))
@Column
public long id;
```

@FieldParam — Binds a field name to replacement parameters inside `@OverrideParams`:

```
@FieldParam(name = "id", params = @Param(name = "startTime", value = "1600000000000"))
```

@OverrideParams — Class-level annotation for subclasses to change generator parameters of inherited fields **without rewriting the @Generator annotation**:

```
@OverrideParams(fields = {
    @FieldParam(name = "id", params = @Param(name = "startTime", value =
"1600000000000")),
    @FieldParam(name = "uid", params = @Param(name = "startTime", value =
"1600000000000"))
})
```

```
public class ChildEntity extends ParentEntity { }
```



`@OverrideParams` is the recommended way to give subclasses **different Snowflake ID spaces** without redeclaring `@Generator`.

3.6. @Mapping — Custom Type Mapping

`@Mapping` overrides Army's default Java→SQL type inference. Army's `_MappingFactory` automatically maps common types (String, Long, BigDecimal, LocalDateTime, etc.) to sensible SQL types. You only need `@Mapping` when:

- Storing binary data in a text column with a specific charset
- Using database-specific types like MySQL `SET` or PostgreSQL `ENUM`
- Mapping collection types (List, Set) to JSON/JSONB
- Overriding the framework's type choice (e.g., forcing `CHAR` for a hash)

3.6.1. Attributes

- `value` (optional) — Fully-qualified class name of the `MappingType` implementation.
- `type` (optional) — `Class` reference of the `MappingType`. **Takes precedence over `value`**.
- `params` (optional) — Extra parameters for the mapping type, e.g., the enum type name for PostgreSQL `ENUM`.
- `func` (optional) — Java method reference for constructing/creating instances of the mapped type. Format: `"ClassName::new"` (constructor) or `"ClassName::methodName"` (static factory). Consumed by `PgSingleRangeType.fromMethod`, `PgMultiRangeType.fromMethod`, and their array variants via `PgRangeType.createRangeFunction`. Resolved by `_ArmyPgRangeType.loadConstructor` / `loadFactoryMethod`.
- `charset` (optional) — Character set name. **Required** when `value` is a `TextMappingType` representing binary data (e.g., `InputStream`, `Path`).
- `elements` (optional) — Element type(s) for collection mappings. Required for `ElementMappingType` implementations like MySQL `SET`. Not needed for array types — the component type is obtained via reflection.

3.6.2. Examples from stock-data

```
// SHA-256 hash → fixed-length CHAR(44) instead of VARCHAR
@Column(precision = 44, comment = "file hash (SHA-256, standard base64)")
@Mapping("io.army.mapping.SqlCharType")
public String fileHash;

// List<Long> → JSONB (PostgreSQL) or JSON (MySQL), defaulting to '[]'
@Column(notNull = true, defaultValue = "[]", comment = "vector ID list")
@Mapping("io.army.mapping.PreferredJsonbType")
```

```
private List<Long> vectorIdList;
```

3.7. @Index — Index Definitions

`@Index` defines a database index within `@Table#indexes`. At runtime, `TableMetaUtils.createIndexList` reads all `@Index` annotations, resolves index names and types, and builds `IndexMeta` objects. If no index covers the `id` field, a unique primary key index is appended automatically.

3.7.1. Attributes

- **name** (required) — Index name. Supports all placeholders. See the naming convention below.
- **fieldList** (optional) — Simple list of Java field names. Each name refers to a field in the domain class and is resolved to its column name at runtime.
- **fields** (optional) — `@IndexField[]` with per-column settings (collation, opclass, sort order). **Takes precedence over fieldList** when both are specified.
- **unique** (optional, default `false`) — Whether the index enforces uniqueness. The primary key index is always unique.
- **type** (optional) — Index access method. Supports `BTREE`, `HASH`, `FULLTEXT`, `SPATIAL` (MySQL); `btree`, `hash`, `gist`, `gin`, `brin`, `hsw` (PostgreSQL). Empty string (default) = database default. Supports `${DEFAULT}`, `${RUNTIME}`, `${OPTIONAL}` placeholders.

3.7.2. Index Name Convention

Placeholder	Generated Name
<code>\${DEFAULT_VALUE}</code>	<code>uni_{table}_{col1}_{col2}</code> (unique) or <code>idx_{table}_{col1}_{col2}</code> (non-unique)
<code>\${DEFAULT}</code>	Same as above, but consults <code>META-INF/army/TableMeta.properties</code> first; falls back to <code>DEFAULT_VALUE</code> behavior
Literal string	Used as-is

3.7.3. Examples from stock-data

```
// Stock: unique composite index – no duplicate (exchange, code) pairs
@Table(name = "stock",
      indexes = @Index(name = "${DEFAULT_VALUE}", unique = true,
                      fieldList = {"exchange", "code"})) // →
uni_stock_exchange_code

// StockQuotes: unique index – one quote record per stock per trading day
@Table(name = "stock_quotes",
      indexes = @Index(name = "${DEFAULT_VALUE}", unique = true,
                      fieldList = {"stockId", "date"})) // →
uni_stock_quotes_stockId_date
```

```
// UploadRecord: two non-unique indexes for fast lookup
@Table(name = "upload_record",
      indexes = {
        @Index(name = "${DEFAULT_VALUE}", fieldList = "userId"), // →
        @Index(name = "${DEFAULT_VALUE}", fieldList = "fileHash") // →
      })
idx_upload_record_userId
idx_upload_record_fileHash
```

3.7.4. @IndexField — Per-Column Index Configuration

For scenarios requiring per-column settings (collation, opclass, sort order), use `@IndexField` instead of simple `fieldList`:

```
// Standard B-tree index with column collation
@Index(name = "${DEFAULT_VALUE}", unique = true, type = "btree",
      fields = {
        @IndexField(name = "exchange"),
        @IndexField(name = "code", collation = "en_US.utf8")
      })

// PostgreSQL HNSW vector index for embedding similarity search
@Index(name = "${DEFAULT_VALUE}", type = "hnsu",
      fields = @IndexField(name = "embedding", opclass = "vector_cosine_ops"))
```

Attribute	Description
<code>name</code> (required)	Java field name of the indexed column
<code>collation</code> (optional)	Column collation, e.g., <code>"en_US.utf8"</code> . PostgreSQL only. Supports <code>\${DEFAULT}</code> and <code>\${OPTIONAL}</code>
<code>opclass</code> (optional)	Operator class for PostgreSQL, e.g., <code>"vector_cosine_ops"</code> , <code>"jsonb_path_ops"</code> . Supports <code>\${DEFAULT}</code> and <code>\${OPTIONAL}</code>
<code>order</code> (optional)	Sort order: <code>SortOrder.ASC</code> / <code>DESC</code> / <code>DEFAULT</code>
<code>nulls</code> (optional)	NULL ordering: <code>NullsOrder.NULLS_FIRST</code> / <code>NULLS_LAST</code> / <code>DEFAULT</code>

3.8. @MappedSuperclass — The Base Domain Pattern

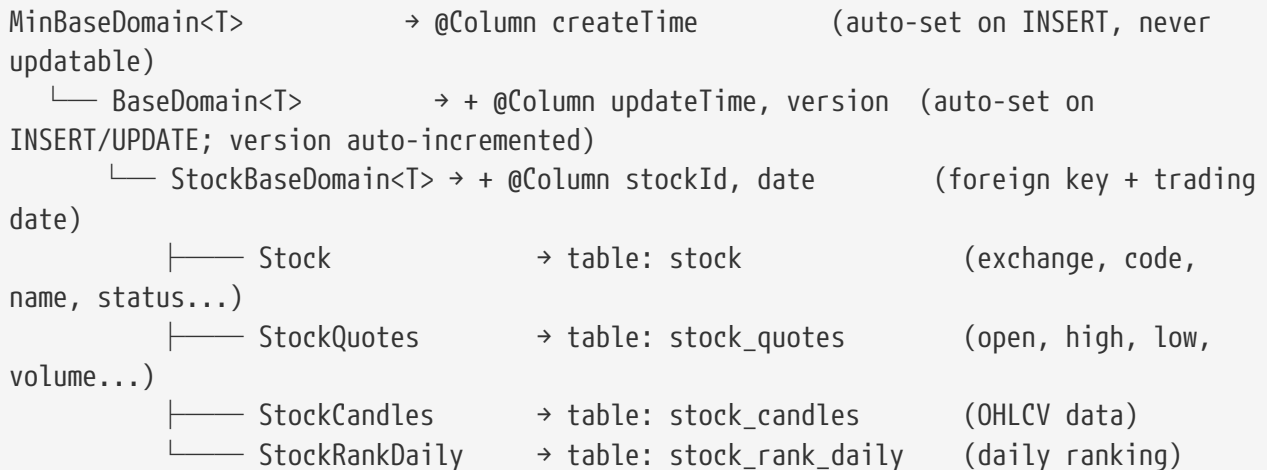
`@MappedSuperclass` marks a class whose `@Column`-annotated fields are **inherited by subclasses**, but which does **not** map to any database table. It's purely a field-template mechanism.

At startup, `TableMetaUtils.mappedClassPair` walks the class hierarchy upward from the domain class: it collects every superclass with `@MappedSuperclass` or `@Table` (reversing the list so the topmost superclass comes first), then `createFieldMetaList` collects all `@Column` fields from each class in order.

Unlike `@Inheritance`, a `@MappedSuperclass` has no discriminator column and cannot be queried directly.

3.8.1. The Inheritance Hierarchy

The `army-example-stock-data` module uses a three-level `@MappedSuperclass` chain:



Only the concrete classes at the bottom have `@Table`. The `@MappedSuperclass` levels above add fields that every concrete entity inherits.

3.8.2. Code Example

```
// Level 1: Every entity needs a creation timestamp
@MappedSuperclass
public abstract class MinBaseDomain<T> extends MinBaseDomain<T>> {

    @Column
    public LocalDateTime createTime; // Auto-set by the framework on INSERT
}

// Level 2: Add update tracking and optimistic locking
@MappedSuperclass
public abstract class BaseDomain<T> extends BaseDomain<T>> extends MinBaseDomain<T> {

    @Column
    public LocalDateTime updateTime; // Auto-refreshed on every INSERT / UPDATE

    @Column
    public int version; // Auto-incremented on UPDATE (optimistic lock)
}

// Level 3: Add fields shared by all stock-related entities
@MappedSuperclass
public abstract class StockBaseDomain<T> extends StockBaseDomain<T>> extends BaseDomain<T> {

    @Column(notNull = true, updatable = false, comment = "stock id")
    public Long stockId;
```

```

    @Column(notNull = true, updatable = false, comment = "trading date")
    public LocalDate date;
}

// Concrete entity: inherits createTime/updateTime/version/stockId/date from the chain
@Table(name = "stock_quotes", comment = "daily stock quotes",
        indexes = @Index(name = "${DEFAULT_VALUE}", unique = true,
                fieldList = {"stockId", "date"}))
public class StockQuotes extends StockBaseDomain<StockQuotes> {

    @Generator(value = "io.army.generator.snowflake.Snowflake8Generator",
            params = @Param(name = "startTime", value = "1779111192831"))
    @Column
    public long id;

    @Column(precision = 9, scale = 3, comment = "open price")
    public BigDecimal open;
    // ... more fields
}

```

3.8.3. Key Points

1. A `@MappedSuperclass` never maps to a table — only `@Table` subclasses do.
2. All `@Column` annotations from every `@MappedSuperclass` in the chain are **inherited** by the concrete subclass.
3. Fields without `@Column` are **not persisted**.
4. You can nest `@MappedSuperclass` to build reusable inheritance chains.
5. For small projects, skip `@MappedSuperclass` and declare all fields directly — it's entirely optional.

3.9. @Inheritance / @DiscriminatorValue — Single-Table Inheritance

While `@MappedSuperclass` shares field definitions, `@Inheritance` implements the **single-table inheritance** strategy: a parent class and its subclasses all live in one database table, with a discriminator column telling them apart.

At startup, `TableMetaUtils.discriminator` locates the field named in `@Inheritance` and verifies it is an `Enum` type. `TableMetaUtils.discriminatorValue` reads `@DiscriminatorValue` on each child class and validates it against the enum constants.

```

// Parent: the root of the inheritance tree
@Table(name = "u_user", comment = "User entity")
@Inheritance("status") // "status" field acts as the discriminator
public abstract class User extends BaseDomain<User> {

```

```

    @Column(comment = "User status")
    public UserStatus status; // Must be an enum implementing CodeEnum
}

// Child: identified by discriminator value "ADMIN"
@DiscriminatorValue("ADMIN")
public class Admin extends User { }

// Child: identified by discriminator value "MEMBER"
@DiscriminatorValue("MEMBER")
public class Member extends User { }

```

The discriminator field must be an `Enum` implementing `io.army.struct.CodeEnum`. The framework uses this column to filter rows by entity subtype in queries.



Only **one** `@Inheritance` is allowed per class hierarchy. If multiple superclasses have `@Inheritance`, Army throws `MetaException`. The parent table with `@Inheritance` cannot be marked as `immutable`.



`@Inheritance` and `@MappedSuperclass` serve different purposes. Use `@MappedSuperclass` when you just want to share field definitions. Use `@Inheritance` when you want polymorphic queries — storing different entity types in one table and filtering by discriminator.

3.10. @DdlMode — DDL Generation Control

Controls whether Army generates `CREATE TABLE` DDL during schema initialization. At runtime, `TableMetaUtils.tableCreateDdl` reads `@Table.ddlMode()`:

Value	Behavior
<code>CREATE</code> (default)	Always generate <code>CREATE TABLE</code> DDL
<code>NONE</code>	Never generate DDL. Use for tables managed externally (e.g., by Flyway or Liquibase)
<code>DEFAULT</code>	Check <code>META-INF/army/TableMeta.properties</code> key <code>{className}.Table.ddlMode</code> ; fallback to <code>CREATE</code>

```

// Prevent DDL generation for a legacy table
@Table(name = "legacy_data", comment = "Legacy data", ddlMode = DdlMode.NONE)
public class LegacyData { }

```

3.11. @Codec — Reserved for Future Use

`@Codec` is a marker annotation with no current runtime effect. The `TableFieldMeta` constructor records its presence in `CODEC_MAP`, but the framework does not act on it yet. It is reserved as an

extension point for future custom encode/decode interceptors.

3.12. TableMeta.properties — Runtime Configuration

`TableMeta.properties` is the **runtime override hub** of Army. Place this file at `META-INF/army/TableMeta.properties` on the classpath (typically `src/main/resources/META-INF/army/TableMeta.properties`). Army scans **all JARs and directories** on the classpath and merges entries from every location — this lets you change annotation attributes without recompiling.

3.12.1. Placeholder Expressions Reference

Many annotation attributes accept placeholder expressions instead of literal values. These placeholders are resolved against `META-INF/army/TableMeta.properties` at startup. They are defined as constants in `TableMetaUtils`:

Placeholder	Constant	Behavior
<code>\${DEFAULT}</code>	<code>TableMetaUtils.DEFAULT_EXP</code>	Check <code>META-INF/army/TableMeta.properties</code> first; if not found, use the framework’s built-in convention (e.g., <code>camelCase</code> → <code>snake_case</code> for column names)
<code>\${RUNTIME}</code>	<code>TableMetaUtils.RUNTIME_EXP</code>	Must be in <code>META-INF/army/TableMeta.properties</code> ; throws <code>MetaException</code> if missing. Use when the value depends on the deployment environment
<code>\${OPTIONAL}</code>	<code>TableMetaUtils.OPTIONAL_EXP</code>	Check <code>META-INF/army/TableMeta.properties</code> first; if not found, silently skip the element (used mainly for index types). Not all attributes support <code>\${OPTIONAL}</code>
<code>\${DEFAULT_VALUE}</code>	<code>TableMetaUtils.DEFAULT_VALUE_EXP</code>	Skip <code>META-INF/army/TableMeta.properties</code> entirely; always use the convention-based default (e.g., auto-generated index names)



`${DEFAULT}` and `${DEFAULT_VALUE}` are NOT the same: * `${DEFAULT}` — allows override via properties, falls back to convention if not found * `${DEFAULT_VALUE}` — bypasses properties entirely, always uses the framework default Use `${DEFAULT_VALUE}` when you never want the value accidentally overridden.

3.12.2. What Happens When Not Configured

When you use a placeholder in your annotation, the framework enters a resolution chain. Here is the **exact fallback behavior** for every attribute — what happens when the property is **not configured** in `META-INF/army/TableMeta.properties`:

Attribute (Placeholder)	Annotation	Behavior When NOT Found
Table name <code>\${DEFAULT}</code>	<code>@Table(name = "\${DEFAULT}")</code>	Convert class simple name to snake_case: `_MetaBridge.camelToLowerCase()` → <code>StockQuotes</code> → <code>"stock_quotes"</code> → <code>UserAccount</code> → <code>"user_account"</code>
	<code>@Table(name = "\${RUNTIME}")</code>	Throw <code>MetaException</code> — no fallback
Table comment <code>\${DEFAULT}</code>	<code>@Table(comment = "\${DEFAULT}")</code>	Convert class simple name to spaced comment: `_MetaBridge.camelToComment()` → <code>StockQuotes</code> → <code>"Stock Quotes"</code> → <code>UserOrder</code> → <code>"User Order"</code>
	<code>@Table(comment = "\${RUNTIME}")</code>	Throw <code>MetaException</code> — no fallback
Catalog / Schema <code>\${DEFAULT}</code>	<code>@Table(catalog = "\${DEFAULT}")</code>	Empty string → database default
	<code>@Table(catalog = "\${RUNTIME}")</code>	Throw <code>MetaException</code> — no fallback
Column name <code>\${DEFAULT}</code>	<code>@Column(name = "\${DEFAULT}")</code>	Convert field name camelCase → snake_case: `_MetaBridge.camelToLowerCase()` → <code>userName</code> → <code>"user_name"</code> → <code>createTime</code> → <code>"create_time"</code>
	<code>@Column(name = "\${RUNTIME}")</code>	Throw <code>MetaException</code> — no fallback
Column comment <code>\${DEFAULT}</code>	<code>@Column(comment = "\${DEFAULT}")</code>	Reserved fields (id/createTime/...) → auto-comment; Enum fields → <code>"enum, @see {EnumClass}"</code> ; Business fields → throw <code>MetaException</code>
	<code>@Column(comment = "\${RUNTIME}")</code>	Throw <code>MetaException</code> — no fallback
	(empty string)	(no comment value)
Same as <code>\${DEFAULT}</code> — reserved → auto, business → error	Column precision <code>DEFAULT_EXP</code>	<code>@Column(precision = Column.DEFAULT_EXP)</code>
By Java type: <code>String</code> → 255, <code>BigDecimal</code> → 16, <code>Enum</code> → max label+5, <code>UUID</code> → 36, <code>Vector</code> → 1024		<code>RUNTIME_EXP</code>
Throw <code>MetaException</code> — no fallback	Column scale <code>DEFAULT_EXP</code>	<code>@Column(scale = Column.DEFAULT_EXP)</code>
By type: <code>BigDecimal</code> → 3, temporal types → 6 (microsecond); otherwise -1		<code>RUNTIME_EXP</code>

Attribute (Placeholder)	Annotation	Behavior When NOT Found
Throw <code>MetaException</code> — no fallback	Column default <code>\${DEFAULT}</code>	<code>@Column(defaultValue = "\${DEFAULT}")</code>
Type-appropriate zero: number → <code>"0"</code> , string → <code>""</code> , boolean → <code>"false"</code> , <code>LocalDate</code> → <code>"1970-01-01"</code> , <code>LocalDateTime</code> → <code>"1970-01-01 00:00:00"</code>		<code>RUNTIME_EXP</code>
Throw <code>MetaException</code> — no fallback	Generator type <code>RUNTIME</code>	<code>@Generator(type = GeneratorType.RUNTIME)</code>
Throw <code>MetaException</code> — no fallback; property must provide <code>Generator.type</code> or <code>GeneratorStrategy</code>		<code>DEFAULT</code>
<code>@Generator(type = GeneratorType.DEFAULT)</code>		Child tables in <code>@Inheritance</code> → <code>POST</code> ; Otherwise → <code>PRECED</code>
<code>PRECED/POST</code>	(literal enum)	Always used as-is — no placeholder involved
Index name <code>\${DEFAULT_VALUE}</code>	<code>@Index(name = "\${DEFAULT_VALUE}")</code>	Always auto-generated (ignores properties): Unique → <code>uni_{table}_{col1}_{col2}</code> Non-unique → <code>idx_{table}_{col1}_{col2}</code>
	<code>\${DEFAULT}</code>	<code>@Index(name = "\${DEFAULT}")</code>
	Falls back to <code>\${DEFAULT_VALUE}</code> behavior (auto-generated name)	<code>\${OPTIONAL}</code>
<code>@Index(name = "\${OPTIONAL}")</code>	<code>null</code> → index is silently skipped	Index type <code>\${DEFAULT}</code>
<code>@Index(type = "\${DEFAULT}")</code>	Empty string → database default index type	
<code>\${RUNTIME}</code>	<code>@Index(type = "\${RUNTIME}")</code>	
Throw <code>MetaException</code> — no fallback	<code>\${OPTIONAL}</code>	<code>@Index(type = "\${OPTIONAL}")</code>
<code>null</code> → entire index is silently skipped	IndexField collation/opclass <code>\${DEFAULT}</code>	<code>@IndexField(collation = "\${DEFAULT}")</code>
Empty string → no collation/opclass		<code>\${RUNTIME}</code>

Attribute (Placeholder)	Annotation	Behavior When NOT Found
<code>@IndexField(collation = "\${RUNTIME}")</code>	Throw <code>MetaException</code> — no fallback	DDL mode DEFAULT

3.12.3. Usage Examples

```
// — Table name: auto-derive from class name, allow override via properties —
// Without properties: StockQuotes → "stock_quotes"
// With properties: com.example.domain.StockQuotes.Table.name=stock_v2
@Table(name = "${DEFAULT}", comment = "stock data")

// — Table name: MUST be in properties, startup fails otherwise —
// Used when team A defines the entity but team B controls deployment name
@Table(name = "${RUNTIME}", comment = "cross-team table")

// — Column name: camelCase → snake_case by default, configurable —
// Without properties: userName → "user_name"
// With properties: com.example.Stock.userName.Column.name=login_name
@Column(name = "${DEFAULT}")
public String userName;

// — Column precision: must be configured at startup – varies by deployment —
// Without properties: throws MetaException
// With properties: com.example.Stock.embedding.Column.precision=768
@Column(precision = Column.RUNTIME_EXP)
public String embedding;

// — Index name: always auto-generate, never override —
// @Table(name = "stock") + @Index(fieldList = "userId")
// → unique: uni_stock_user_id, non-unique: idx_stock_user_id
@Index(name = "${DEFAULT_VALUE}", fieldList = "userId")

// — Index name: default generation but allow override via properties —
// Without properties: same as ${DEFAULT_VALUE}
// With properties: com.example.IndexMeta[0].name=my_index
@Index(name = "${DEFAULT}", fieldList = "userId")

// — Optional index: skip entirely on staging, keep in production —
// Without properties: index is silently skipped
// With properties: com.example.IndexMeta[0].type=btree → index is created
@Index(name = "idx_stock", type = "${OPTIONAL}", fieldList = "userId")

// — Column default value: type-appropriate zero —
// LocalDateTime → '1970-01-01 00:00:00'
// BigDecimal → 0
// String → ''
@Column(defaultValue = "${DEFAULT}", comment = "upload complete time")
public LocalDateTime uploadCompleteTime;
```

```
// —— Generator type: defer to deployment config ——
// Development: POST (auto-increment), Production: PRECEDE (Snowflake)
@Generator(type = GeneratorType.RUNTIME)
@Column
public long id;
```

3.12.4. Property Key Format

Every property key follows this pattern:

```
{fully.qualified.ClassName}.{fieldName|Table}.{AttributeCategory}.{attributeName}
```

Category	Example Key	Example Value
Table	com.example.domain.Stock.Table.name	stock_v2
Table.comment	com.example.domain.Stock.Table.comment	stock data v2
Table.catalog	com.example.domain.Stock.Table.catalog	production
Table.schema	com.example.domain.Stock.Table.schema	public
Table.ddlMode	com.example.domain.Stock.Table.ddlMode	NONE
Column.name	com.example.domain.Stock.userName.Column.name	user_name
Column.comment	com.example.domain.Stock.status.Column.comment	custom status
Column.precision	com.example.domain.Stock.name.Column.precision	512
Column.scale	com.example.domain.Stock.offerPrice.Column.scale	4
Column.defaultValue	com.example.domain.Stock.status.Column.defaultValue	'ACTIVE'
Column.collation	com.example.domain.Stock.name.Column.collation	en_US.utf8
Generator.type	com.example.domain.Stock.id.Generator.type	POST
GeneratorStrategy	com.example.domain.Stock.id.GeneratorStrategy	io.army.generator.Snowflake8GeneratorStrategy:{ "startTime":1779111192831 }
Mapping.value	com.example.domain.Stock.id.Mapping.value	io.army.mapping.SqlBigIntType
IndexMeta[n].name	com.example.domain.Stock.IndexMeta[0].name	idx_stock_custom
IndexMeta[n].type	com.example.domain.Stock.IndexMeta[0].type	btree
IndexMeta[n].field.X.collation	com.example.domain.Stock.IndexMeta[0].field.code.collation	en_US.utf8
IndexMeta[n].field.X.opclass	com.example.domain.Stock.IndexMeta[0].field.embedding.opclass	vector_cosine_ops



Property keys are **case-sensitive**. Keyword components — `Table`, `Column`, `Generator`, `GeneratorStrategy`, `Mapping`, `IndexMeta` — must appear **exactly** with that capitalization.



The `IndexMeta[n]` syntax uses a 0-based index: `[0]` refers to the **first** `@Index` in `@Table.indexes` array, `[1]` to the second, etc. The `field.{fieldName}` segment uses the **Java field name** (not the column name).

The `GeneratorStrategy` value format is `fully.qualified.StrategyClass[:jsonParams]`. The optional JSON parameters after the colon are parsed by the strategy's static `create(String)` factory method.

3.12.5. Complete Configuration Example

Here is a realistic `META-INF/army/TableMeta.properties` file showing all categories in one place:

```
# File: META-INF/army/TableMeta.properties
# Place in: src/main/resources/META-INF/army/TableMeta.properties

# —— @Table ——
com.example.domain.Stock.Table.name=stock_v2
com.example.domain.Stock.Table.comment=stock data v2
com.example.domain.Stock.Table.schema=public
com.example.domain.Stock.Table.ddlMode=NONE

# —— @Column ——
com.example.domain.Stock.userName.Column.name=user_name
com.example.domain.Stock.status.Column.comment=custom status
com.example.domain.Stock.name.Column.precision=512
com.example.domain.Stock.offerPrice.Column.scale=4
com.example.domain.Stock.status.Column.defaultValue='ACTIVE'
com.example.domain.Stock.name.Column.collation=en_US.utf8

# —— @Generator ——
com.example.domain.Stock.id.Generator.type=POST
com.example.domain.Stock.id.GeneratorStrategy=io.army.generator.Snowflake8GeneratorStrategy:{"startTime":1779111192831}

# —— @Mapping ——
com.example.domain.Stock.id.Mapping.value=io.army.mapping.SqlBigIntType

# —— @Index ——
com.example.domain.Stock.IndexMeta[0].name=idx_stock_custom
com.example.domain.Stock.IndexMeta[0].type=btree

# Skip an optional index (empty value = skip)
com.example.domain.Stock.IndexMeta[1].type=

# —— @IndexField (per-column index settings) ——
```

```
com.example.domain.Stock.IndexMeta[0].field.code.collation=en_US.utf8
com.example.domain.Stock.IndexMeta[0].field.embedding.opclass=vector_cosine_ops
```

```
# —— @DdlMode ——
com.example.domain.LegacyData.Table.ddlMode=NONE
```



Start with an empty `TableMeta.properties` file and add entries only when you need to override defaults for a specific deployment environment. Most projects need only 2–5 entries (e.g., switching generator types, adjusting precisions, disabling DDL).

4. MappingType — Java-to-Database Type Mapping

Every domain field in Army must declare its **type mapping** — the rule that converts between a Java type and a database column type. This mapping is defined either implicitly (via default type inference) or explicitly (via the `@Mapping` annotation). The `MappingType` interface is the core abstraction that orchestrates this conversion.

4.1. Overview: MappingType Interface

```
public sealed interface MappingType permits AbstractMappingType, ArrayMappingType {

    /// The Java type that this mapping represents.
    Class<?> javaType();

    /// Resolve to the concrete SQL data type for the current database dialect.
    DataType map(ServerMeta meta);

    /// Convert a Java value → SQL parameter value (before binding).
    Object beforeBind(DataType dataType, MappingEnv env, Object source);

    /// Convert a database value → Java object (after fetching).
    Object afterGet(DataType dataType, MappingEnv env, Object source);

    /// Return the corresponding array MappingType for this scalar type.
    MappingType arrayTypeOfThis();
}
```

A `MappingType` implementation defines four things:

1. **Which Java type** it handles — returned by `javaType()`.
2. **Which SQL type** it maps to — resolved by `map(ServerMeta)`, which is database-dialect-aware.
3. **How values are serialized** before binding to a prepared statement — `beforeBind()`.

4. How values are deserialized after fetching from a `ResultSet` — `afterGet()`.

For example, `io.army.mapping.StringType` maps `java.lang.String` → MySQL `VARCHAR` / PostgreSQL `VARCHAR` / Oracle `VARCHAR2`, and during `beforeBind()` it can convert `Number`, `Boolean`, `Enum`, `LocalDateTime`, etc. into their string representations automatically.

4.2. Default Type Inference

When a domain field has **no** `@Mapping` annotation, Army infers the `MappingType` from the field's Java type via `io.army.mapping._MappingFactory.getDefaultIfMatch()`.

4.2.1. Built-in Java-to-MappingType Lookup

Java Type	MappingType	SQL Type (PostgreSQL / MySQL)
<code>String</code>	<code>StringType</code>	<code>VARCHAR</code> / <code>VARCHAR</code>
<code>boolean</code> / <code>Boolean</code>	<code>BooleanType</code>	<code>BOOLEAN</code> / <code>TINYINT(1)</code>
<code>int</code> / <code>Integer</code>	<code>IntegerType</code>	<code>INTEGER</code> / <code>INT</code>
<code>long</code> / <code>Long</code>	<code>LongType</code>	<code>BIGINT</code> / <code>BIGINT</code>
<code>float</code> / <code>Float</code>	<code>FloatType</code>	<code>REAL</code> / <code>FLOAT</code>
<code>double</code> / <code>Double</code>	<code>DoubleType</code>	<code>DOUBLE PRECISION</code> / <code>DOUBLE</code>
<code>short</code> / <code>Short</code>	<code>ShortType</code>	<code>SMALLINT</code> / <code>SMALLINT</code>
<code>byte</code> / <code>Byte</code>	<code>ByteType</code>	<code>SMALLINT</code> / <code>TINYINT</code>
<code>char</code> / <code>Character</code>	<code>SqlCharType</code>	<code>CHAR(1)</code> / <code>CHAR(1)</code>
<code>BigInteger</code>	<code>BigIntegerType</code>	<code>NUMERIC</code> / <code>DECIMAL</code>
<code>BigDecimal</code>	<code>BigDecimalType</code>	<code>NUMERIC</code> / <code>DECIMAL</code>
<code>LocalDateTime</code>	<code>LocalDateTimeType</code>	<code>TIMESTAMP</code> / <code>DATETIME</code>
<code>LocalDate</code>	<code>LocalDateType</code>	<code>DATE</code> / <code>DATE</code>
<code>LocalTime</code>	<code>LocalTimeType</code>	<code>TIME</code> / <code>TIME</code>
<code>OffsetDateTime</code>	<code>OffsetDateTimeType</code>	<code>TIMESTAMPZ</code> / (n/a)
<code>ZonedDateTime</code>	<code>ZonedDateTimeType</code>	<code>TIMESTAMPZ</code> / <code>TIMESTAMP</code>
<code>Instant</code>	<code>InstantType</code>	<code>TIMESTAMPZ</code> / <code>TIMESTAMP</code>
<code>Year</code>	<code>YearType</code>	<code>SMALLINT</code> / <code>SMALLINT</code>
<code>YearMonth</code>	<code>YearMonthType</code>	<code>DATE</code> / <code>DATE</code>
<code>MonthDay</code>	<code>MonthDayType</code>	<code>DATE</code> / <code>DATE</code>
<code>UUID</code>	<code>UUIDType</code>	<code>UUID</code> / <code>VARCHAR(36)</code>
<code>byte[]</code>	<code>VarBinaryType</code>	<code>BYTEA</code> / <code>VARBINARY</code>
<code>BitSet</code>	<code>BitSetType</code>	<code>VARBIT</code> / <code>BIT</code>
<code>ZoneId</code>	<code>ZoneIdType</code>	<code>VARCHAR</code> / <code>VARCHAR</code>

4.2.2. Enum Type Inference

When a domain field's Java type is an `enum`, Army has three distinct mapping strategies:

Strategy	Implements/Extensions	Stored in DB as	Use Case
<code>NameEnumType</code>	(plain <code>enum</code> , no marker interface)	<code>VARCHAR</code> (or PostgreSQL native <code>ENUM</code> with <code>@DefinedType</code>)	Simple enums where <code>name()</code> is human-readable enough (e.g., <code>NORMAL</code> , <code>DELISTED</code>)
<code>TextEnumType</code>	implements <code>TextEnum</code>	<code>VARCHAR</code> (or PostgreSQL native <code>ENUM</code> with <code>@DefinedType</code>)	Enum values need a display text different from the Java constant name (e.g., <code>ZORO</code> → <code>"Roronoa Zoro"</code>)
<code>CodeEnumType</code>	implements <code>CodeEnum</code>	<code>INTEGER</code>	Fixed, stable integer codes for maximum storage efficiency and ABI compatibility

NameEnumType — Default Enum Strategy

A plain `enum` with no marker interface uses `NameEnumType`. The stored value is `Enum.name()`, which is the Java constant identifier.

```
public enum StockStatus {
    NORMAL,      // → stored as "NORMAL"
    DELISTED,    // → stored as "DELISTED"
    STOPT,       // → stored as "STOPT"
    UNKNOWN      // → stored as "UNKNOWN"
}

@Column(notNull = true, defaultValue = "'NORMAL'", comment = "????????")
public StockStatus status;

// Default inference: StockStatus → NameEnumType → VARCHAR column
```

Under the hood, `NameEnumType.beforeBind()` calls `((Enum<?>) source).name()`, and `afterGet()` calls `Enum.valueOf(enumClass, string)`. The mapping to SQL types is dialect-dependent:

- **MySQL**: native `ENUM('NORMAL', 'DELISTED', 'STOPT', 'UNKNOWN')`
- **PostgreSQL**: `VARCHAR` by default; becomes a native `CREATE TYPE ... AS ENUM` if the enum class is annotated with `@DefinedType`
- **H2**: native `ENUM`
- **SQLite**: `VARCHAR`



A plain enum mapped via `NameEnumType` is safe for adding new constants at the end of the list, but **renaming** or **reordering** existing constants will break existing data

(the stored string won't match).

TextEnumType — Text-Based Enum with Custom Label

`NameEnumType` stores `Enum.name()` — but `name()` is constrained to valid Java identifiers. What if you need the database to store a more descriptive text, like "Roronoa Zoro" instead of "ZORO"?

That's what `TextEnumType` is for. By implementing `io.army.struct.TextEnum`, the enum provides a `text()` method that returns the value to be stored:

```
public enum QinArmy implements TextEnum {

    ZORO("Roronoa Zoro"),          // stored as "Roronoa Zoro"
    ANZAI("Mitsuyoshi Anzai");    // stored as "Mitsuyoshi Anzai"

    private final String text;

    QinArmy(String text) {
        this.text = text;
    }

    @Override
    public String text() {
        return this.text;
    }
}

@Column(comment = "???")
public QinArmy swordsman; // → TextEnumType, stored as VARCHAR
```

`TextEnumType.beforeBind()` calls `((TextEnum) source).text()` instead of `name()`, and `afterGet()` reads the stored text and looks up the enum instance via an internal `Map<String, TextEnum>` built at initialization time.

`TextEnumType` vs `NameEnumType`:

Aspect	<code>NameEnumType</code>	<code>TextEnumType</code>
Stored value source	<code>Enum.name()</code> (Java constant identifier)	<code>TextEnum.text()</code> (arbitrary string)
Rename safety	Renaming the Java constant breaks data	Renaming the Java constant is safe — only <code>text()</code> matters
Database readability	Stored as Java-style identifiers (NORMAL)	Stored as human-readable text ("Roronoa Zoro")
Special enum support	Handles <code>Month</code> and <code>DayOfWeek</code> (see <code>SpecialEnums</code>)	No special enum handling
<code>@DefinedType</code> support	❑ (PostgreSQL native ENUM)	❑ (PostgreSQL native ENUM)

CodeEnumType — Integer-Code Enum

When the enum values have stable, semantically-meaningful integer codes, `CodeEnumType` is the best choice. It stores only the integer, which is the most storage-efficient option and is immune to refactoring (the code stays unchanged even if the Java constant is renamed).

```
public enum BankAccountType implements CodeEnum {

    BANK(0),
    LENDER(100),
    BORROWER(200),
    PARTNER(300),
    LENDER_BUSINESS(400),
    BORROWER_BUSINESS(500),
    GUARANTOR(600);

    private final short code;

    BankAccountType(short code) {
        this.code = code;
    }

    @Override
    public final int code() {
        return this.code;
    }
}

@Column(comment = " ")
public BankAccountType accountType; // → CodeEnumType, stored as INT
```

`CodeEnumType.beforeBind()` returns `((CodeEnum) source).code()` as an `Integer`. `afterGet()` is flexible — it can deserialize from `Integer`, `Long`, `Short`, `Byte`, `BigInteger`, or `String`.

`CodeEnumType` is mapped to an `INTEGER` column on all dialects. It is the most robust strategy for long-lived enums whose semantics are encoded in numbers (e.g., protocol codes, status codes, wire-format constants).



Never use consecutive integer codes (0, 1, 2, 3, ...) for `CodeEnum`. Always leave gaps between codes — use increments of **10** (e.g., 0, 10, 20, 30) or **100** (e.g., 0, 100, 200, 300). Consecutive values make it impossible to insert a new constant between existing ones without renumbering, which would corrupt existing data. With spaced codes, you can always insert a new value — e.g., between `LENDER(100)` and `BORROWER(200)` you can add `LENDER_PREMIUM(150)`.



Never implement both `CodeEnum` and `TextEnum` on the same enum class. Army will reject it at startup with an `IllegalArgumentException`.

PostgreSQL Native ENUM via @DefinedType

Both `NameEnumType` and `TextEnumType` support PostgreSQL native `ENUM` types through the `io.army.struct.DefinedType` annotation. When applied to the enum class, Army generates a `CREATE TYPE` DDL and maps the field to the named PostgreSQL enum type instead of `VARCHAR`:

```
@DefinedType(name = "stock_status_enum", category = TypeCategory.ENUM)
public enum StockStatus {
    NORMAL,
    DELISTED,
    STOPT,
    UNKNOWN
}

@Column(comment = "股票状态")
public StockStatus status;

// Without @DefinedType → VARCHAR
// With @DefinedType(name = "stock_status_enum") → PostgreSQL: CREATE TYPE
stock_status_enum AS ENUM (...)
```

This is especially useful when the enum type needs to be shared across multiple tables, or when you want the database to enforce value constraints at the schema level.

Special Enums: Month and DayOfWeek

`NameEnumType` has built-in intelligent handling for `java.time.Month` and `java.time.DayOfWeek`. These enums are commonly returned as integers by database functions (e.g., `MONTH()`, `DAYOFWEEK()`) and as strings by others (`MONTHNAME()`, `DAYNAME()`).

Month — `NameEnumType.afterGet()` accepts:

- `Integer` (1–12) → `Month.of(n)` — covers MySQL `MONTH()` / PostgreSQL `EXTRACT(MONTH FROM ...)`
- `Long` (1–12) → `Month.of((int) n)`
- `String` ("JANUARY", "FEBRUARY", ...) → `Month.valueOf(name.toUpperCase())` — covers `MONTHNAME()`
- `LocalDate`, `YearMonth`, `MonthDay`, `LocalDateTime`, `OffsetDateTime`, `ZonedDateTime` → `Month.from(temporal)`

```
// MySQL: SELECT MONTH(create_time) m, MONTHNAME(create_time) mn FROM ...
// Both m (integer) and mn (string) can be read into a Month field
@Column
public Month birthMonth; // → NameEnumType + Month special handling
```

DayOfWeek — `NameEnumType.afterGet()` accepts:

- `Integer` — on MySQL, 1 maps to `SUNDAY` (MySQL `DAYOFWEEK()` convention); otherwise `DayOfWeek.of(n)`

- `Long` → same as integer
- `String("MONDAY", "TUESDAY", ...)` → `DayOfWeek.valueOf(name.toUpperCase())`
- `LocalDate, LocalDateTime, OffsetDateTime, ZonedDateTime` → `DayOfWeek.from(temporal)`

```
// MySQL: SELECT DAYOFWEEK(create_time) dow, DAYNAME(create_time) dn FROM ...
// In MySQL, DAYOFWEEK returns 1=Sunday, 2=Monday, ..., 7=Saturday
// NameEnumType handles this MySQL-specific mapping automatically
@Column
public DayOfWeek tradingDay; // → NameEnumType + DayOfWeek special handling
```

4.3. Navigation: stock-data Example

The `army-example-stock-data` module contains a complete stock market application demonstrating all mapping patterns discussed in this chapter. Browse its `domain/` package to see real `@Mapping` usages across `Stock`, `StockQuotes`, `StockChatConversation`, `UploadRecord`, and vector store entities.

4.4. @Mapping Annotation: Explicit Mapping Control

When the default inference is insufficient — or when you need a non-standard database type — use the `io.army.annotation.Mapping` annotation.

4.4.1. Annotation Definition

```
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Mapping {

    /// Fully-qualified class name of the MappingType implementation.
    /// Used as a string to avoid compile-time dependencies on dialect-specific
    classes.
    String value() default "";

    /// The MappingType class itself (takes precedence over value()).
    Class<?> type() default void.class;

    /// Extra configuration parameters (e.g., PostgreSQL enum type name).
    String[] params() default {};

    /// Java method reference for constructing or creating instances of the mapped
    type.
    /// Format: "ClassName::new" (constructor) or "ClassName::methodName" (static
    factory).
    /// Consumed by PgSingleRangeType.fromMethod / PgMultiRangeType.fromMethod and
    array variants
    /// via PgRangeType.createRangeFunction; resolved by
    _ArmyPgRangeType.loadConstructor / loadFactoryMethod.
```

```

    /// For non-range types, dispatched via _MappingFactory.doMap.
    String func() default "";

    /// Character set for text-based binary mappings (e.g., LongBlobType +
    InputStream).
    String charset() default "";

    /// Element types for collection mappings (e.g., MySQL SET).
    Class<?>[] elements() default {};
}

```

4.4.2. Resolution Priority

The `@Mapping` value follows the same placeholder mechanism as other annotations:

1. If `type()` is set (not `void.class`), it takes precedence over `value()`.
2. If `value()` is `"${DEFAULT}"`, Army checks `TableMeta.properties`; if not configured, falls back to default type inference.
3. If `value()` is `"${RUNTIME}"`, the property **must** be configured in `TableMeta.properties` or a `MetaException` is thrown.
4. Otherwise, `value()` is a fully-qualified class name resolved via `Class.forName()`.

4.4.3. Example: Mapping a String to TEXT (Long Content)

The `StockChatConversation.firstContent` field stores a Java `String` but needs a PostgreSQL `TEXT` column (not `VARCHAR`). This is achieved by explicitly specifying `TextType`:

```

@Column(notNull = true, updatable = false, comment = " ")
@Mapping("io.army.mapping.TextType")
private String firstContent;

```

Similarly, `SpringAiChatMemory.content` and `SpringAiVectorStore.content` both use `@Mapping("io.army.mapping.TextType")` to store potentially long message text in a `TEXT` column:

```

// From SpringAiChatMemory (chat memory module)
@Column(name = "${DEFAULT}", notNull = true, comment = "${DEFAULT}")
@Mapping("io.army.mapping.TextType")
private String content;

// From SpringAiVectorStore (vector store module)
@Column(name = "${DEFAULT}", comment = "${DEFAULT}")
@Mapping("io.army.mapping.TextType")
private String content;

```

4.4.4. Example: Mapping a String to CHAR (Fixed-Length)

The `UploadRecord.fileHash` field stores a SHA-256 Base64 hash (always 44 characters). Using `SqlCharType` enforces a `CHAR(44)` column:

```
@Column(precision = 44, comment = "SHA-256, base64")
@Mapping("io.army.mapping.SqlCharType")
public String fileHash;
```

4.4.5. Example: Runtime-Configurable Mapping

The `SpringAiChatVectorStore.conversationId` uses `"${DEFAULT}"` to allow deployment-time type selection:

```
/// Mapping#value() should be one of below:
/// - SqlBigIntType
/// - StringType
/// - UUIDType
@Column(name = "${DEFAULT}", nullable = true, precision = Column.DEFAULT_EXP)
@Mapping("${DEFAULT}")
private String conversationId;
```

In `TableMeta.properties`, you can choose:

```
# Use Long-based conversation ID
com.example.stock.domain.StockChatVectorStore.conversationId.Mapping.value=io.army.mapping.SqlBigIntType

# Or use UUID-based
#
com.example.stock.domain.StockChatVectorStore.conversationId.Mapping.value=io.army.mapping.UUIDType
```

4.5. Array Types

Army supports PostgreSQL native arrays (`integer[]`, `text[]`, etc.) through the `array` subpackage of `io.army.mapping`. Each scalar `MappingType` has a corresponding array variant.

4.5.1. Default Array Inference

When a Java array field has no `@Mapping`, the framework infers the array `MappingType` from the element type via `_MappingFactory.getDefaultIfMatch()`:

- `int[] / Integer[]` → `IntegerArrayType` → PostgreSQL `INTEGER[]`
- `long[] / Long[]` → `LongArrayType` → PostgreSQL `BIGINT[]`

- `String[]` → `StringArrayType` → PostgreSQL `TEXT[]`
- `BigDecimal[]` → `BigDecimalArrayType` → PostgreSQL `NUMERIC[]`
- `LocalDate[]` → `LocalDateArrayType` → PostgreSQL `DATE[]`
- `Boolean[]` → `BooleanArrayType` → PostgreSQL `BOOLEAN[]`

```
// Auto-inferred: Integer[] → IntegerArrayType → INTEGER[]
@Column(name = "tag_ids")
private Integer[] tagIds;
```

4.5.2. Custom Array Mapping

For custom element types (enums, composites), use `@Mapping` to specify an array type explicitly:

```
// PostgreSQL ENUM array – element type inferred from field component type via
// reflection
@Mapping("io.army.mapping.array.NameEnumArrayType")
private Gender[] genders;
```

4.6. Composite Type (PostgreSQL Row Types)

`io.army.mapping.CompositeType` maps POJOs annotated with `@DefinedType` to PostgreSQL [composite \(row\) types](#). The POJO's fields (annotated with `@Column`) become the composite type's attributes, serialized as `(val1, val2, ...)`.

`io.army.struct.DefinedType#fieldOrder()` is **mandatory** because PostgreSQL composite types require a fixed attribute order. The parser must know which position each field occupies in the tuple to correctly serialize and deserialize. Without it, Army cannot determine the mapping between Java field names and tuple positions.



Additionally:

- `@DefinedType.name()` defines the PostgreSQL composite type name (e.g., `currency_pair`). Army uses it for DDL generation and type resolution.
- The POJO must provide a no-arg constructor or a constructor matching field types for deserialization.
- `CompositeType` is PostgreSQL-only. For arrays of composite types, use `io.army.mapping.array.CompositeArrayType`.

First, define the composite type POJO:

```
/// @see io.army.struct.DefinedType
@DefinedType(name = "currency_pair", fieldOrder = {"code", "rate"})
public final class CurrencyPair {
```

```

@Column
private final String code;

@Column
private final BigDecimal rate;

public CurrencyPair(String code, BigDecimal rate) {
    this.code = code;
    this.rate = rate;
}

public String code() {
    return this.code;
}

public BigDecimal rate() {
    return this.rate;
}

@Override
public int hashCode() {
    return Objects.hash(this.code, this.rate);
}

@Override
public boolean equals(Object obj) {
    final boolean match;
    if (obj == this) {
        match = true;
    } else if (obj instanceof CurrencyPair o) {
        match = this.code.equals(o.code) && this.rate.equals(o.rate);
    } else {
        match = false;
    }
    return match;
}
}

```

Then reference `CompositeType` in the `@Mapping` annotation:

```

@Table(name = "exchange_rate")
public class ExchangeRate {

    @Id
    @Column
    @GeneratedValue
    private Long id;

```

```

@Column
@Mapping(io.army.mapping.CompositeType.class) // or
"io.army.mapping.CompositeType"
private CurrencyPair pair;

}

```

Serialization example:

- **beforeBind**: a `CurrencyPair("USD", 7.25)` becomes the literal `('USD',7.25)`
- **afterGet**: the text `('USD',7.25)` is parsed back to `CurrencyPair("USD", 7.25)`

4.7. Vector Type (pgvector Embeddings)

For AI/ML embedding storage, Army provides `io.army.mapping.optional.VectorType` — a `MappingType` for the PostgreSQL `pgvector` extension's `VECTOR` type.

```

public final class VectorType extends _ArmyNoInjectionType
    implements MappingType.SqlExtension {

    public static VectorType from(Class<?> javaType) {
        // Only float[] is supported
        return INSTANCE;
    }

    @Override
    public DataType map(ServerMeta meta) {
        if (meta.serverDatabase() != Database.PostgreSQL) {
            throw mapError(this, meta); // VECTOR is PostgreSQL-only
        }
        return PgType.VECTOR;
    }

    // beforeBind: float[] → "[0.1,0.2,...,0.9]" SQL string
    // afterGet:   SQL string → float[]
}

```

4.7.1. Vector Store Example (army-spring-ai-vector-store)

The `io.army.spring.ai.vectorstore.SpringAiVectorStore` base class uses `VectorType` for its embedding column:

```

/// Java type: float[] → MappingType: VectorType → PostgreSQL: VECTOR
@Column(name = "${DEFAULT}", notNull = true,
        precision = Column.DEFAULT_EXP, // required: embed dimension configured at
runtime
        comment = "${DEFAULT}")

```

```
@Mapping("io.army.mapping.optional.VectorType")
private float[] embedding;
```



The `precision = Column.DEFAULT_EXP` (which equates to `Integer.MIN_VALUE`) signals that the vector dimension must be configured at runtime. Without `precision` configured (via `TableMeta.properties` or the `@Column` annotation), Army throws a `MetaException`.

4.7.2. Vector Store Domain Subclass Example

Domain entities extend the parent class to define their table and indexes:

```
// Long-term chat memory vector store – uses HNSW index with cosine similarity
@Table(name = "stock_chat_vector_store",
    indexes = {
        @Index(name = "${DEFAULT_VALUE}", type = "hnsw",
            fields = @IndexField(name = "embedding",
                opclass = "vector_cosine_ops")),
        @Index(name = "${DEFAULT_VALUE}", fieldList = "conversationId"),
        @Index(name = "${DEFAULT_VALUE}", type = "gin", fieldList =
"metadata")
    },
    comment = "████████████████")
public class StockChatVectorStore extends SpringAiChatVectorStore {
}

// Document vector store – uses HNSW index with L2 (Euclidean) distance
@Table(name = "stock_document_vector_store",
    indexes = {
        @Index(name = "${DEFAULT_VALUE}", type = "hnsw",
            fields = @IndexField(name = "embedding",
                opclass = "vector_l2_ops")),
        @Index(name = "${DEFAULT_VALUE}", unique = true, fieldList =
"documentId"),
        @Index(name = "${DEFAULT_VALUE}", type = "gin", fieldList =
"metadata")
    },
    comment = "██████████")
public class StockDocumentVectorStore extends SpringAiVectorStore {
}
```

4.8. JSON / JSONB Types

Army provides several JSON mapping types for document-oriented storage:

MappingType	Java Types	Behavior
JsonType	String, Map, List, Set, POJO	Always maps to vanilla JSON (MySQL JSON, PostgreSQL JSON)
PreferredJsonbType	String, Map, List, Set, POJO	Maps to JSONB on PostgreSQL, falls back to JSON on MySQL
JsonbType	String, Map, List, Set, POJO	Strictly maps to JSONB (PostgreSQL-only, throws on other dialects)

`PreferredJsonbType` is the most commonly used because it provides optimal storage and indexing (`jsonb`) on PostgreSQL while gracefully degrading to `json` on MySQL.

4.8.1. Metadata Storage (army-spring-ai-vector-store)

The `io.army.spring.ai.vectorstore.SpringAiVectorStore.metadata` field uses `PreferredJsonbType` to store a `Map<String, Object>`:

```
@Column(name = "${DEFAULT}", nullable = true, defaultValue = "{}", comment =
"${DEFAULT}")
@Mapping("io.army.mapping.PreferredJsonbType")
private Map<String, Object> metadata;
```

On PostgreSQL this produces a `JSONB` column with a `GIN` index for efficient containment queries; on MySQL it produces a `JSON` column.

4.8.2. Specialized Data Storage (army-spring-ai-model-chat-memory)

The `io.army.spring.ai.chat.memory.SpringAiChatMemory.specializedData` field stores tool call data or tool response data as JSON:

```
/// store one of below :
/// 1. AssistantMessage.getToolCalls()
/// 2. ToolResponseMessage.getResponses()
@Column(name = "${DEFAULT}", nullable = true, defaultValue = "[]",
comment = "${DEFAULT}")
@Mapping("io.army.mapping.PreferredJsonbType")
private String specializedData;
```

The `ArmyMessageChatMemory.add()` method uses `JsonCodec` to serialize tool calls/responses before inserting:

```
// Inside ArmyMessageChatMemory.copyMessageInfo()
if (message instanceof AssistantMessage a) {
    o.setSpecializedData(codec.encode(a.getToolCalls()));
} else if (message instanceof ToolResponseMessage t) {
    o.setSpecializedData(codec.encode(t.getResponses()));
}
```

```
}
```

4.8.3. List Storage in JSONB (UploadRecord)

The `io.army.example.stock.domain.UploadRecord.vectorIdList` field stores a `List<Long>` of vector IDs as JSONB:

```
@Column(notNull = true, defaultValue = "[]", comment = "[]ID[]")
@Mapping("io.army.mapping.PreferredJsonbType")
private List<Long> vectorIdList;
```

4.9. Text vs Varchar — Choosing the Right String Mapping

Army distinguishes between three string storage strategies:

MappingType	SQL Type	Use Case
<code>StringType</code> (default for <code>String</code>)	<code>VARCHAR(precision)</code>	Short to medium strings (names, codes, hashes) — precision comes from <code>@Column(precision=N)</code>
<code>TextType</code>	<code>TEXT</code>	Long, unbounded text (article content, message body, document text)
<code>SqlCharType</code>	<code>CHAR(N)</code>	Fixed-length strings (SHA-256 Base64 hashes, ISO country codes)

```
// VARCHAR – default inference for String, precision from @Column
@Column(notNull = true, precision = 130, comment = "[]")
public String name; // → StringType → VARCHAR(130)

// CHAR – fixed-length, used for deterministic-length hashes
@Column(precision = 44, comment = "[]")
@Mapping("io.army.mapping.SqlCharType")
public String fileHash; // → SqlCharType → CHAR(44)

// TEXT – unbounded, for long content
@Column(notNull = true, comment = "[]")
@Mapping("io.army.mapping.TextType")
private String firstContent; // → TextType → TEXT
```

4.10. How TypeMappingHandler Resolves to SQL Types

After Army determines a field's `MappingType`, the `io.army.dialect.TypeMappingHandler` interface finalizes the concrete `DataType` for the target database:

```
public sealed interface TypeMappingHandler permits TypeMappingHandlerSupport {

    /// For a given custom type name and MappingType array, return the concrete
    DataType.
    DataType apply(String typeName, MappingType[] typeArray, int index);
}
```

The implementation `io.army.dialect.TypeMappingHandlerSupport` performs this resolution:

1. **Custom type lookup** — First checks if a `DefinedTypeMapFunc` is registered (for user-defined database types like `PostgreEnum`).
2. **Named type lookup** — Checks the dialect's internal type registry (e.g., `"vector"` → `PgType.VECTOR`).
3. **Array detection** — If the type name contains `[` (e.g., `"integer[]"`), it resolves the element type then wraps it via `arrayTypeOfThis()`.
4. **Fallback** — If no match is found, returns the `defaultType` (the `MappingType.map()` result).

```
// Simplified resolution logic in TypeMappingHandlerSupport
final MappingType handleDefinedType(final String typeName, final MappingType
defaultType) {
    // 1. Try user-defined type function
    MappingType type = func.apply(typeName, serverMeta, typeMap);
    if (type != null) return type;

    // 2. Check if it's an array type (e.g., "integer[]")
    final int index = typeName.indexOf('[');
    if (index < 0) {
        type = this.typeMap.getOrDefault(typeName, defaultType);
    } else {
        type = this.typeMap.get(typeName.substring(0, index)); // element type
        if (type != null) {
            type = ((MappingType.SqlArray) type.arrayTypeOfThis()).unlimited();
        } else {
            type = defaultType;
        }
    }
    return type;
}
```

This design means the `MappingType` hierarchy is database-agnostic — each `MappingType` declares its target `DataType` per-dialect in `map()`, and `TypeMappingHandler` provides the final resolution for custom/composite/array types. In most cases, developers interact only with the `@Mapping` annotation; the handler chain operates transparently behind the scenes.

4.11. Custom MappingType — Extending the Type System

When the built-in mapping types don't fit your domain, Army provides two abstract base classes for creating custom types.

4.11.1. Step 1: Choose the Right Base Class

Scenario	Base Class	Example
Non-array Java type	Extend <code>io.army.mapping.UserMappingType</code>	A custom <code>Currency</code> type mapped to <code>NUMERIC(10,2)</code>
Array Java type	Extend <code>io.army.mapping.UserArrayType</code>	A custom <code>IntRange[]</code> type mapped to PostgreSQL <code>INT4RANGE[]</code>

`UserMappingType` extends `AbstractMappingType`, which already provides `beforeBind()` / `afterGet()` defaults (pass-through to `String`). `UserArrayType` extends `UserMappingType` and additionally implements `ArrayMappingType`.

4.11.2. Step 2: Implement the Corresponding `Sql*` Marker Interface

Every mapping type should declare what **kind** of SQL value it represents by implementing one of `MappingType`'s inner marker interfaces. The Army framework uses these markers for validation and type compatibility checks.



The `map()`, `beforeBind()`, and `afterGet()` methods work regardless of whether you implement a `Sql*` marker — the mapping will execute correctly in basic read/write scenarios.

The problem arises when Army needs to know the **semantic category** of your type. For example:

- Generating DDL — Army must decide `INT` vs `VARCHAR` vs `TIMESTAMP` column type. Without a marker, it cannot infer the SQL type family.
- Type compatibility checks — when comparing two columns or assigning a parameter, Army verifies that both sides are in the same type family (e.g., both are `SqlNumber`).
- Array element inference — when creating an array type automatically, Army uses the marker to determine the element SQL type.

In short: your type **will** work without a marker, but it may fail in DDL generation or type-checking scenarios. Implementing the correct `Sql*` interface is strongly recommended.

Common marker interfaces:

Category	Interface	When to Use
Number	<code>MappingType.SqlInteger</code>	Maps to <code>INT</code> . Your type handles integer values.
Number	<code>MappingType.SqlDecimal</code>	Maps to <code>NUMERIC</code> . Your type handles decimal values.
String	<code>MappingType.SqlText</code>	Maps to <code>TEXT</code> . Your type handles long unbounded strings.
String	<code>MappingType.SqlString</code>	Maps to <code>VARCHAR</code> . Your type handles short bounded strings.
Binary	<code>MappingType.SqlBlob</code>	Maps to <code>BYTEA</code> / <code>BLOB</code> . Your type handles binary data.
JSON	<code>MappingType.SqlJson</code>	Maps to <code>JSON</code> . Your type stores JSON documents.
Extension	<code>MappingType.SqlVector</code>	Maps to pgvector <code>VECTOR</code> type. Combine with <code>MappingType.SqlExtension</code> to auto-create the extension.
Time	<code>MappingType.SqlLocalDateTime</code>	Maps to <code>TIMESTAMP</code> . Your type handles date-time values.



The full list of marker interfaces is in `io.army.mapping.MappingType`:

- **Boolean** — `SqlBoolean`
- **Numeric** — `SqlInteger`, `SqlFloat`, `SqlDecimal`
- **String** — `SqlString`, `SqlText`
- **Binary** — `SqlBinary`, `SqlBlob`, `SqlBit`
- **JSON** — `SqlJson`, `SqlJsonb`
- **Time** — `SqlLocalDate`, `SqlLocalTime`, `SqlLocalDateTime`, `SqlOffsetDateTime`, `SqlDuration`
- **Extension** — `SqlVector`, `SqlExtension`

Pick the one that most closely describes your target SQL type — each has a single unambiguous semantic meaning.

4.11.3. Step 3: Follow the Implementation Pattern

All built-in mapping types follow a consistent pattern. Your custom type must adhere to it for framework compatibility:

```
/// Custom mapping: maps a Java 'Currency' record to PostgreSQL 'NUMERIC(10,2)'.
public final class CurrencyType extends UserMappingType    //
io.army.mapping.UserMappingType
    implements MappingType.SqlDecimal {
```

```

// 1. Static factory method – required for @Mapping resolution
public static CurrencyType from(final Class<?> javaType) {
    if (javaType != Currency.class) {
        throw errorJavaType(CurrencyType.class, javaType);
    }
    return INSTANCE;
}

public static final CurrencyType INSTANCE = new CurrencyType();

// 2. Constructor must be non-public (private or package-private)
private CurrencyType() {
}

// 3. Declare the Java type this mapping handles
@Override
public Class<?> javaType() {
    return Currency.class;
}

// 4. Map to the concrete SQL DataType per database
@Override
public DataType map(final ServerMeta meta) {
    final DataType dataType;
    switch (meta.serverDatabase()) {
        case PostgreSQL:
            dataType = PgType.NUMERIC;
            break;
        case MySQL:
            dataType = MySQLType.DECIMAL;
            break;
        default:
            throw mapError(this, meta);
    }
    return dataType;
}

// 5. Serialize Java value → SQL parameter
@Override
public BigDecimal beforeBind(DataType dataType, MappingEnv env, Object source) {
    if (!(source instanceof Currency c)) {
        throw paramError(this, dataType, source, null);
    }
    return c.value();
}

// 6. Deserialize database value → Java object
@Override
public Currency afterGet(DataType dataType, MappingEnv env, Object source) {
    final Currency value;

```

```

        if (source instanceof Currency c) {
            value = c;
        } else if (source instanceof BigDecimal bd) {
            value = new Currency(bd);
        } else if (source instanceof String s) {
            value = new Currency(new BigDecimal(s));
        } else {
            throw dataAccessError(this, dataType, source, null);
        }
        return value;
    }

    // 7. Required: override hashCode/equals for singleton types
    @Override
    public int hashCode() {
        return System.identityHashCode(this);
    }

    @Override
    public boolean equals(Object obj) {
        return obj instanceof CurrencyType;
    }
}

```



The `from(Class)` static factory method is how `_MappingFactory.doMap` reflectively creates your `MappingType`. Without it, the framework cannot instantiate your type. The constructor must be non-public — all built-in types use `private` constructors and expose a singleton `INSTANCE`.

4.11.4. Array Type Example

For array types, extend `UserArrayType` and implement `MappingType.SqlArray`:

```

/// Custom mapping: maps `Currency[]` to PostgreSQL `NUMERIC[]`.
public final class CurrencyArrayType extends UserArrayType
    implements MappingType.SqlArray {

    public static CurrencyArrayType from(final Class<?> javaType) {
        final CurrencyArrayType instance;
        if (javaType == Currency[].class) {
            instance = LINEAR;
        } else if (javaType == Object.class) {
            instance = UNLIMITED;
        } else if (!javaType.isArray()) {
            throw errorJavaType(CurrencyArrayType.class, javaType);
        } else if (ArrayUtils.underlyingComponent(javaType) == Currency.class) {
            instance = new CurrencyArrayType(javaType);
        } else {
            throw errorJavaType(CurrencyArrayType.class, javaType);
        }
    }
}

```

```

    }
    return instance;
}

/// Used for default array parsing when the database returns an array
/// of unknown dimensions. Typically consumed by {@code
io.army.dialect.TypeMappingHandler}
/// implementations.
public static final CurrencyArrayType UNLIMITED = new
CurrencyArrayType(Object.class);

public static final CurrencyArrayType LINEAR = new
CurrencyArrayType(Currency[].class);

private final Class<?> javaType;

/// private constructor
private CurrencyArrayType(Class<?> javaType) {
    this.javaType = javaType;
}

@Override
public final Class<?> javaType() {
    return this.javaType;
}

@Override
public final Class<?> underlyingJavaType() {
    return Currency.class;
}

@Override
public final DataType map(final ServerMeta meta) {
    final DataType dataType;
    switch (meta.serverDatabase()) {
        case PostgreSQL:
            dataType = PgType.NUMERIC_ARRAY;
            break;
        default:
            throw mapError(this, meta);
    }
    return dataType;
}

@Override
public final MappingType elementType() {
    final MappingType instance;
    final Class<?> javaType = this.javaType;
    if (javaType == Object.class) { // unlimited dimension array
        instance = this;
    } else if (javaType == Currency[].class) {

```

```

        instance = CurrencyType.INSTANCE;
    } else {
        instance = from(javaType.getComponentType());
    }
    return instance;
}

@Override
public MappingType arrayTypeOfThis() throws CriteriaException {
    final Class<?> javaType = this.javaType;
    if (javaType == Object.class) { // unlimited dimension array
        return this;
    }
    return from(ArrayUtils.arrayClassOf(javaType));
}

// beforeBind / afterGet omitted for brevity – see built-in array types for
reference

@Override
public int hashCode() {
    return this.javaType.hashCode();
}

@Override
public boolean equals(Object obj) {
    final boolean match;
    if (obj == this) {
        match = true;
    } else if (obj instanceof CurrencyArrayType o) {
        match = o.javaType == this.javaType;
    } else {
        match = false;
    }
    return match;
}
}

```



For array types, the element type is obtained from the field's component type via reflection — you do NOT need `@Mapping(elements = ...)`. The `elements` attribute is only for generic collections (`List<Foo>`, `Set<Bar>`) where type erasure loses the element type.

4.11.5. Quick Checklist

Before writing your custom type, verify:

- ☐ Non-array type → extends `UserMappingType`; Array type → extends `UserArrayType`
- ☐ Implements the correct `Sql*` marker interface (e.g., `SqlDecimal`, `SqlVector`)

- Provides a `public static ... from(Class)` factory method
- Constructor is `private` (or package-private)
- Overrides `javaType()`, `map()`, `beforeBind()`, `afterGet()`
- Overrides `hashCode()` and `equals(Object)`
- For array types, also overrides `underlyingJavaType()`, `elementType()`, and `arrayTypeOfThis()`
- (Optional) If the type needs a PostgreSQL extension, implements `MappingType.SqlExtension`

4.12. Full Mapping Chain Summary

Stage	What Happens	Example
1. Annotation Processing	Army reads <code>@Mapping</code> (or infers default) to determine <code>MappingType</code>	<code>@Mapping("io.army.mapping.optional.VectorType") → VectorType</code>
2. Dialect Resolution	<code>MappingType.map(ServerMeta)</code> returns database-specific <code>DataType</code>	<code>VectorType.map() → PgType.VECTOR</code> (PostgreSQL only)
3. TypeHandler Finalization	<code>TypeMappingHandler.apply()</code> resolves custom/composite/array types	<code>"vector" → PgType.VECTOR;</code> <code>"integer[]" → PgType.INTEGER_ARRAY</code>
4. Value Binding	<code>MappingType.beforeBind()</code> serializes Java value to SQL parameter	<code>VectorType.beforeBind():</code> <code>float[]{0.1,0.2} → "[0.1,0.2]"</code>
5. Value Fetching	<code>MappingType.afterGet()</code> deserializes database value to Java object	<code>VectorType.afterGet(): "[0.1,0.2]"</code> <code>→ float[]{0.1,0.2}</code>

5. Criteria API — Writing SQL

The Criteria API is Army's SQL construction layer. It generates a database-agnostic statement tree through a type-safe Java DSL, which the dialect parser then translates into native SQL.

5.1. Standard Criteria API

The Standard Criteria API is Army's database-agnostic SQL construction layer. It generates an internal statement tree through a type-safe Java DSL, which the dialect parser then translates into native SQL for your target database.

Concept	Interface (design)	Implementation
Entry point	<code>SQLs</code> (utility class)	<code>criteria/impl/SQLs.java</code>
Statement specs	<code>standard/StandardInsert</code> <code>standard/StandardQuery</code> <code>standard/StandardUpdate</code> <code>standard/StandardDelete</code>	<code>impl/StandardInserts</code> <code>impl/StandardQueries</code> <code>impl/StandardUpdates</code> <code>impl/StandardDeletes</code>
Standard functions	<code>standard/SQLFunction</code>	<code>impl/Functions</code>

Concept	Interface (design)	Implementation
Window functions	<code>dialect/Window</code>	<code>impl/Windows</code>

5.1.1. The SQLs Utility Class

`SQLs` (`../army-core/src/main/java/io/army/criteria/impl/SQLs.java`) is the unified entry point. Internally, its static factory methods delegate to `StandardInserts`, `StandardQueries`, `StandardUpdates`, and `StandardDeletes`:

```
// SELECT entry
Select stmt = SQLs.query()...asQuery();

// INSERT entry
Insert stmt = SQLs.singleInsert()...asInsert();

// UPDATE entry
Update stmt = SQLs.singleUpdate()...asUpdate();

// DELETE entry
Delete stmt = SQLs.singleDelete()...asDelete();

// Domain UPDATE entry (no table alias required)
Update stmt = SQLs.domainUpdate()...asUpdate();

// Domain DELETE entry
Delete stmt = SQLs.domainDelete()...asDelete();
```

Each returns a type-safe chain that exposes only the clauses valid for that statement type.

5.1.2. SELECT Statements

`SQLs.query()` enters the SELECT chain. Internally it calls `StandardQueries#simpleQuery`, which returns a `StandardQuery.WithSpec`.

Basic SELECT

```
final Select stmt;
stmt = SQLs.query()
    .select(Stock_.id, Stock_.name, Stock_.offerPrice)
    .from(Stock_.T, AS, "s")
    .where(Stock_.name.equal("Tesla"))
    .and(Stock_.createTime.less(LocalDateTime.now().minusDays(1)))
    .limit(1)
    .asQuery();
```

```
SELECT s.id, s.name, s.offer_price
FROM stock AS s
WHERE s.name = ? -- param: 'Tesla'
AND s.create_time < ? -- param: now - 1 day
LIMIT 1
```

Full Clause Chain

The SELECT chain follows SQL clause order:

1. **WITH** ... **AS** — CTE prefix (via `with(name).as(...)`)
2. **SELECT** — `select(...)` / `selectDistinct(...)`
3. **FROM** — `from(table, AS, alias)` / `nested from(function)`
4. **JOIN** — `join(...)` / `leftJoin(...)` / `rightJoin(...)` / `crossJoin(...)`
5. **WHERE** — `where(predicate).and(predicate)...`
6. **GROUP BY** — `groupBy(...)`
7. **HAVING** — `having(predicate)`
8. **WINDOW** — `window(name).as(...)` (named window definitions)
9. **ORDER BY** — `orderBy(...)`
10. **LIMIT / OFFSET** — `limit(n).offset(n)`
11. **FOR UPDATE** — `lock(...)` for row-level locking
12. **UNION / INTERSECT / EXCEPT** — set operations via `union(...)`, etc.
13. `asQuery()` — terminal method, returns `Select`

Each clause method returns the next narrow interface in the chain, so the compiler enforces correct SQL clause ordering.

5.1.3. INSERT Statements

Single Table INSERT

`SQLs.singleInsert()` returns `StandardInsert._PrimaryOptionSpec<Insert>`. Internally it calls `StandardInserts#singleInsert`.

Domain mode — pass domain objects directly. Defined in `InsertStatement._DomainValuesClause`
`InsertStatement._DomainValueClause`
`_ValuesColumnDefaultSpec`:

```
// (A) Single domain object
// <TS extends T> R value(TS domain) → returns _DmlInsertClause<Insert>
final Insert stmt1;
stmt1 = SQLs.singleInsert()
```

```

        .insertInto(Stock_.T)                                // → _ColumnListSpec
        .value(new Stock("GOOGL", "Alphabet"))              // →
_DmlInsertClause<Insert>
        .asInsert();                                         // → Insert

// (B) List of domain objects (multi-row)
//     <TS extends T> R values(List<TS> domainList)
final Insert stmt2;
stmt2 = SQLs.singleInsert()
        .insertInto(Stock_.T)
        .defaultValue(Stock_.status, StockStatus.NORMAL)   // optional column default
        .values(stockList)                                   // →
_DmlInsertClause<Insert>
        .asInsert();

```

Generated SQL (PostgreSQL)

```

-- (A) Single domain: columns follow field declaration order in the document's Stock:
--     id(PRECEDE), create_time, update_time, version, exchange,
--     code, name, status, offer_price, listing_date
INSERT INTO stock (id, create_time, update_time, version, exchange,
                  code, name, status, offer_price, listing_date)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

-- (B) List with defaultValue: status per-row;
--     when Stock.status is null, 'NORMAL' is substituted
INSERT INTO stock (id, create_time, update_time, version, exchange,
                  code, name, status, offer_price, listing_date)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?),
       (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

```

Method	Interface signature	Returns
<code>value(TS domain)</code>	<code><TS extends T> R value(TS domain) in _DomainValueClause<T,R></code>	<code>_DmlInsertClause<I> → .asInsert()</code>
<code>values(List<TS>)</code>	<code><TS extends T> R values(List<TS> domainList) in _DomainValuesClause<T,R></code>	<code>_DmlInsertClause<I> → .asInsert()</code>

Value mode — Static (single row / multi-row)

Requires `.values()` (no-arg) first — this bridges `_ColumnListSpec` via `_StaticValuesClause` to `StandardInsert._StandardValuesParensClause`, which extends `InsertStatement._ValuesParensClause`.

Each `parens()` call adds one VALUES row. All columns are referenced via `FieldMeta` constants.

Single row:

```

// .values() → _StandardValuesParensClause
// .parens(Consumer<_StaticValueSpaceClause>) → _ValuesParensCommaSpec → .asInsert()

```

```

final Insert stmt;
stmt = SQLs.singleInsert()
    .insertInto(Stock_.T)
    .values() //
    _StaticValuesClause.values() no-arg
    .parens(row -> row //
Consumer<_StaticValueSpaceClause<Stock>>
    .space(Stock_.exchange, "NYSE") // space(FieldMeta<T>,
@Nullable Object)
    .comma(Stock_.code, "AAPL") // comma(FieldMeta<T>,
@Nullable Object)
    .comma(Stock_.name, SQLs::param, "Alphabet")// comma(FieldMeta,
BiFunction, E) – binding
    .ifComma(Stock_.offerPrice, offerValue) // conditional comma –
skipped when null
    )
    .asInsert();

```

Generated SQL (PostgreSQL)

```

-- Value mode always inserts ALL table columns; fields not explicitly set
-- use DEFAULT (NullMode.DEFAULT→INSERT_DEFAULT for PostgreSQL)
-- id, create_time, update_time, version are managed by Army
INSERT INTO stock (id, create_time, update_time, version, exchange,
                  code, name, status, offer_price, listing_date)
VALUES (?, ?, ?, ?, ?, ?, ?, DEFAULT, ?,          -- offer_price uses DEFAULT when
offerValue is null
        DEFAULT)          -- listing_date uses DEFAULT
-- params: id(snowflake), 'NYSE', 'AAPL', ?(param: 'Alphabet'), offerValue

```

Table 1. *_StaticValueSpaceClause* methods:

Method	Signature
<code>space(field, value)</code>	First pair; <code>value</code> can be literal, <code>Expression</code> , <code>null</code> , or <code>SQLs.DEFAULT</code>
<code>space(field, funcRef, value)</code>	First pair with binding: <code>SQLs::param</code> , <code>SQLs::literalValue</code> , etc.
<code>spaceIf(field, value)</code>	Conditional first pair — skipped when <code>value</code> is <code>null</code>
<code>spaceIf(field, funcRef, value)</code>	Conditional first pair with binding
<code>comma(field, value)</code>	Subsequent pair
<code>comma(field, funcRef, value)</code>	Subsequent pair with binding
<code>ifComma(field, value)</code>	Conditional subsequent pair
<code>ifComma(field, funcRef, value)</code>	Conditional subsequent pair with binding

Multi-row — chain via `.comma()`:

```

// .parens() → _ValuesParensCommaSpec → .comma() loops back
// _ValuesParensCommaSpec extends _CommaClause<_StandardValuesParensClause>
final Insert stmt;
stmt = SQLs.singleInsert()
    .insertInto(Stock_.T)
    .values()
    .parens(r -> r                                     // row 1
        .space(Stock_.exchange, "NYSE")
        .comma(Stock_.code, "AAPL")
        .comma(Stock_.name, "Apple Inc.")
    )
    .comma()                                           // → back to
_StandardValuesParensClause
    .parens(r -> r                                     // row 2
        .space(Stock_.exchange, "NASDAQ")
        .comma(Stock_.code, "GOOGL")
        .comma(Stock_.name, SQLs::param, "Alphabet")
    )
    .comma()
    .parens(r -> r                                     // row 3 (conditional first
column)
        .spaceIf(Stock_.exchange, exchangeValue)    // skipped if exchangeValue
== null
        .comma(Stock_.code, "MSFT")
        .comma(Stock_.name, "Microsoft")
    )
    .asInsert();

```

Generated SQL (PostgreSQL)

```

-- Default mode: all table columns appear in the INSERT list.
-- id, create_time, update_time, version managed by Army;
-- unset nullable fields → DEFAULT (NullMode.DEFAULT→INSERT_DEFAULT for PostgreSQL)
INSERT INTO stock (id, create_time, update_time, version, exchange,
                  code, name, status, offer_price, listing_date)
VALUES (?, ?, ?, ?, ?, ?, ?, DEFAULT, DEFAULT,
        DEFAULT),          -- row 1: 'NYSE', 'AAPL', 'Apple Inc.'
(?, ?, ?, ?, ?, ?, ?, DEFAULT, DEFAULT,
        DEFAULT),          -- row 2: 'NASDAQ', 'GOOGL', ? (param: 'Alphabet')
(?, ?, ?, ?, ?, ?, ?, DEFAULT, DEFAULT,
        DEFAULT)           -- row 3: exchangeValue non-null→param, null→DEFAULT;
'MSFT', 'Microsoft'

```

Value mode — Dynamic (multi-row via **ValuesConstructor**)

Defined in `InsertStatement._DynamicValuesClause`: `values(Consumer<ValuesConstructor<T>>)`. Returns `_DmlInsertClause<I>`.

class="extern"><code>ValuesConstructor<T></code> extends <code>_ValuesParensClause<T></code>. Each <code>parens()</code> returns a comma clause; <code>.comma()</code> chains back to <code>ValuesConstructor</code>.

Inside `parens()`, there are **two** sub-modes:

Static row — type-safe via `FieldMeta` (same `_StaticValueSpaceClause` as above):

```
final Insert stmt;
stmt = SQLs.singleInsert()
    .insertInto(Stock_.T)
    .values(rows -> rows //
Consumer<ValuesConstructor<Stock>>
    .parens(r -> r //
Consumer<_StaticValueSpaceClause<Stock>>
    .space(Stock_.exchange, "NYSE") // space(FieldMeta<T>,
Object)
    .comma(Stock_.code, "AAPL")
    .comma(Stock_.name, "Apple Inc.")
    )
    .comma() // → back to
ValuesConstructor
    .parens(SQLs.SPACE,s-> { // row 2 dynamic column
        s.set(Stock_.exchange, "NASDAQ");
        s.set(Stock_.code, "GOOGL");
        s.set(Stock_.name, SQLs::param, "Alphabet"); // set with
binding
    })
    )
    .asInsert();
```

Generated SQL (PostgreSQL)

```
INSERT INTO stock (id, create_time, update_time, version, exchange,
                  code, name, status, offer_price, listing_date)
VALUES (?, ?, ?, ?, ?, ?, ?, DEFAULT, DEFAULT,
        DEFAULT),      -- row 1: 'NYSE', 'AAPL', 'Apple Inc.'
        (?, ?, ?, ?, ?, ?, ?, DEFAULT, DEFAULT,
        DEFAULT)      -- row 2: 'NASDAQ', 'GOOGL', ? (param: 'Alphabet')
```

Dynamic row — via `Assignments` (uses `set()` instead of `space()/comma()`):

```
final Insert stmt;
stmt = SQLs.singleInsert()
    .insertInto(Stock_.T)
    .values(rows -> rows //
Consumer<ValuesConstructor<Stock>>
    .parens(s -> s // Consumer<Assignments<Stock>>
```

```

        .space(Stock_.exchange, "NYSE")           // space(FieldMeta<T>,
@Nullable Object)
        .comma(Stock_.code, "AAPL")              // each set() returns
Assignments<T>
        .comma(Stock_.name, "Apple Inc.")
        .ifComma(Stock_.offerPrice, offer)       // conditional –
skipped when null
    )
    .comma()
    .parens(SQLs.SPACE,s-> { // row 2  dynamic column
        s.set(Stock_.exchange, "NASDAQ");
        s.set(Stock_.code, "GOOGL");
        s.set(Stock_.name, SQLs::param, "Alphabet"); // set with
binding
    })
    )
    .asInsert();

```

Generated SQL (PostgreSQL)

```

INSERT INTO stock (id, create_time, update_time, version, exchange,
                  code, name, status, offer_price, listing_date)
VALUES (?, ?, ?, ?, ?, ?, ?, DEFAULT, ?,           -- row 1: offer_price uses DEFAULT
when offer is null
        DEFAULT),
        (?, ?, ?, ?, ?, ?, ?, DEFAULT, DEFAULT,
        DEFAULT)
-- row 1: 'NYSE', 'AAPL', 'Apple Inc.', offer
-- row 2: 'NASDAQ', 'GOOGL', ? (param: 'Alphabet')

```

`Assignments` extends `InsertStatement._StaticAssignmentSetClause`:

Method	Signature
<code>set(field, value)</code>	<code>R set(FieldMeta<T>, @Nullable Object)</code> — literal, Expression, or null
<code>set(field, funcRef, value)</code>	<code><E> R set(FieldMeta<T>, BiFunction<...,Expression>, @Nullable E)</code> — with binding
<code>ifSet(field, value)</code>	<code>R ifSet(FieldMeta<T>, @Nullable Object)</code> — conditional, skipped when null
<code>ifSet(field, funcRef, value)</code>	<code><E> R ifSet(FieldMeta<T>, BiFunction<...,Expression>, @Nullable E)</code> — conditional with binding



The *Static* row sub-mode uses the same `StaticValueSpaceClause` lambdas as the non-Dynamic path. The *_Dynamic* row sub-mode uses `Assignments.set()` which is useful when columns are added in a loop or conditionally.

Query mode — insert from a subquery. Defined in `InsertStatement._QueryInsertSpaceClause` `InsertStatement._ColumnListParensClause`.

`_ColumnListParensClause` has two `parens()` overloads for specifying the target column list. `_QueryInsertSpaceClause` has three `space()` overloads for providing the subquery source.

Signal	Method	Returns
static columns	<code>parens(Consumer<_StaticColumnSpaceClause<T>>)</code>	<code>_ComplexColumnDefaultSpec</code>
dynamic columns	<code>parens(SQLs.SymbolSpace, Consumer<Consumer<FieldMeta<T>>>)</code>	<code>_ComplexColumnDefaultSpec</code>
pre-built subquery	<code>space(Supplier<SubQuery>)</code>	<code>_DmlInsertClause<I></code>
inline via function	<code>space(Function<SelectSpec, _DmlInsertClause<I>>)</code>	<code>_DmlInsertClause<I></code>
bare <code>space()</code>	<code>T space()</code>	<code>SelectSpec<_DmlInsertClause<I>> → chain .asInsert()</code>

Form 1 — Static columns + pre-built SubQuery:

```
// _StaticColumnSpaceClause: space(f1,f2) – 1/2/4 column params supported
final Insert stmt;
stmt = SQLs.singleInsert()
    .insertInto(Stock_.T)
    .parens(cols -> cols //
Consumer<_StaticColumnSpaceClause<Stock>>
    .space(Stock_.exchange, Stock_.code) // space(FieldMeta,
FieldMeta) → _StaticColumnDualClause
    .comma(Stock_.name) // comma(FieldMeta)
    )
    .space(subQuery) // Supplier<SubQuery> →
_DmlInsertClause
    .asInsert();
```

Generated SQL (PostgreSQL)

```
INSERT INTO stock (exchange, code, name, id, create_time,
                  update_time, version)
SELECT ... -- subQuery SQL; Army-managed fields (id, create_time, update_time,
version) use DEFAULT
```

Form 2 — Dynamic columns + pre-built SubQuery:

```
// Dynamic column list: each accept(field) adds one column
final Insert stmt;
stmt = SQLs.singleInsert()
    .insertInto(Stock_.T)
    .parens(SQLs.SPACE, list -> { //
Consumer<Consumer<FieldMeta<Stock>>>
    list.accept(Stock_.exchange); // accept(FieldMeta<T>)
```

```

        list.accept(Stock_.code);
        list.accept(Stock_.name);
    })
    .space(subQuery)                // Supplier<SubQuery>
    .asInsert();

```

Generated SQL (PostgreSQL)

```

INSERT INTO stock (exchange, code, name, id, create_time,
                  update_time, version)
SELECT ... -- subQuery SQL; Army-managed fields (id, create_time, update_time,
version) use DEFAULT

```

Form 3 — No column list (subquery determines columns):

```

// Direct .space(subQuery) on _ComplexColumnDefaultSpec – no parens() needed
final Insert stmt;
stmt = SQLs.singleInsert()
    .insertInto(Stock_.T)
    .space(subQuery)                //
    _QueryInsertSpaceClause.space(Supplier)
    .asInsert();

```

Generated SQL (PostgreSQL)

```

INSERT INTO stock
SELECT ... -- subQuery determines all columns

```

Form 4 — No-arg `space()` returns `SelectSpec` (advanced):

```

// space() no-arg → SelectSpec<_DmlInsertClause<Insert>>
// Build subquery inline, chain ends with .asInsert()
final Insert stmt;
stmt = SQLs.singleInsert()
    .insertInto(Stock_.T)
    .parens(cols -> cols.space(Stock_.exchange, Stock_.code, Stock_.name,
Stock_.status))
    .space()                        // T space() → SelectSpec
    .select(Stock_.exchange, Stock_.code, Stock_.name)
    .from(NasdaqTable_.T, AS, "n")
    .where(NasdaqTable_.listed.equal(true))
    .asInsert();

```

Generated SQL (PostgreSQL)

```

INSERT INTO stock (exchange, code, name, status)
SELECT n.exchange, n.code, n.name, DEFAULT

```

```
FROM nasdaq_table AS n
WHERE n.listed = ? -- param: true
```

Child INSERT (Parent-Child Hierarchy)

When using `@Inheritance` single-table inheritance, a **parent insert** must be followed by a **child insert**. This is enforced at the interface level: `insertInto(ParentTableMeta)` returns a chain that ends with `child()` to enter the child table's insert.

Internally, `StandardInserts` handles this via `_ChildInsertIntoClause` and `InsertStatement._ParentInsert20`:

```
// Parent insert → child insert chain
final Insert stmt;
stmt = SQLs.singleInsert()
    .insertInto(ParentUser_.T)          // ParentTableMeta – returns
    _ParentInsert20
    .defaultValue(ParentUser_.status, UserStatus.ADMIN)
    .values(parentUserList)
    .child()                          // enters child insert context
    .insertInto(Admin_.T)              // ChildTableMeta – returns _ColumnListSpec
    .values(adminUserList)
    .asInsert();
```

Generated SQL (PostgreSQL)

```
-- Statement 1: parent rows (no discriminator column)
-- Columns follow field declaration order in ParentUser class
INSERT INTO u_user (id, name, create_time, update_time, version)
VALUES (?, ?, ?, ?, ?),
...

-- Statement 2: child rows (includes discriminator column)
-- Columns follow field declaration order in Admin class;
-- discriminator 'ADMIN' from @DiscriminatorValue is auto-injected
INSERT INTO u_user (id, name, status, create_time, update_time, version)
VALUES (?, ?, ?, ?, ?, ?),
...

-- NOTE: When id GeneratorType is POST (database auto-increment),
-- the id column is NOT included in the column list.
```



The `child()` method is only available when the previous `insertInto()` received a `ParentTableMeta`. Army generates separate `INSERT` statements for parent and child tables internally.

INSERT Options

Option	Interface (from <code>InsertStatement</code>)	Chain position
<code>literalMode(mode)</code>	<code>_PreferLiteralClause</code> — reduce placeholders for large batches	before <code>insertInto()</code>
<code>nullMode(mode)</code>	<code>_NullOptionClause</code> — control NULL handling	before <code>insertInto()</code>
<code>migration()</code>	<code>_MigrationOptionClause</code> — migration mode for schema transfer	before <code>insertInto()</code>
<code>ignoreReturnIds()</code>	<code>_IgnoreReturnIdsOptionClause</code> — skip ID return for conflict clauses	before <code>insertInto()</code>

5.1.4. UPDATE Statements

Single Table UPDATE

`SQLs.singleUpdate()` enters the UPDATE chain. Internally it calls `StandardUpdates#singleUpdate`.

```
final Update stmt;
stmt = SQLs.singleUpdate()
    .update(Stock_.T, AS, "s")           // SingleTableMeta – requires alias
    .set(Stock_.name, "Tesla")
    .set(Stock_.offerPrice, SQLs::plusEqual, addPrice)
    .where(Stock_.id.between(map.get("firstId"), SQLs.AND, map.get("secondId")))
    .asUpdate();
```

Generated SQL (PostgreSQL)

```
UPDATE stock AS s
SET s.name = ?,
    s.offer_price = s.offer_price + ?,
    s.update_time = ?,
    s.version = s.version + 1
WHERE s.id BETWEEN ? AND ?
```

The `set()` method comes from `UpdateStatement._StaticSetClause`. It supports value operators like `SQLs::plusEqual`, `SQLs::minusEqual`, etc., and `setRow()` for multi-column subquery assignment.

Domain UPDATE

`SQLs.domainUpdate()` is for updating domain objects directly — no table alias needed. Internally it calls `StandardUpdates#simpleDomain`, which implements `StandardUpdate._DomainUpdateClause`:

```
final Update stmt;
stmt = SQLs.domainUpdate()
    .update(Stock_.T, "s")               // TableMeta – no AS keyword
    .set(Stock_.name, "Tesla")
```

```
.where(Stock_.id.equal(100L))
.asUpdate();
```

Generated SQL (PostgreSQL)

```
UPDATE stock s
SET s.name = ?,
    s.update_time = ?,
    s.version = s.version + 1
WHERE s.id = ? -- param: 100
```

Entry Point	Description
<code>SQLs.singleUpdate()</code>	Standard UPDATE with table alias; <code>update(SingleTableMeta, AS, alias)</code>
<code>SQLs.domainUpdate()</code>	Domain UPDATE without <code>AS</code> keyword; <code>update(TableMeta, alias)</code> — supports <code>ChildTableMeta</code>

Batch UPDATE

For batch operations with named parameters:

```
final BatchUpdate stmt;
stmt = SQLs.batchSingleUpdate()
    .update(Stock_.T, AS, "s")
    .sets(pairs -> pairs
        .setSpace(Stock_.name, SQLs::namedParam) // named param
placeholder
        .setSpace(Stock_.offerPrice, SQLs::namedLiteral) // named literal
placeholder
    )
    .where(Stock_.id.spaceEqual(SQLs::namedParam)) // space* methods for
named params
    .asUpdate();

// Execute with named parameter list
List<Map<String, Object>> paramList = List.of(
    Map.of("name", "Tesla", "offerPrice", 250, "id", 1),
    Map.of("name", "Apple", "offerPrice", 180, "id", 2)
);
long rows = session.update(stmt.namedParamList(paramList));
```

Generated SQL (PostgreSQL)

```
-- Batch 1: name='Tesla', offerPrice=250, id=1
UPDATE stock AS s
SET s.name = ?,
    s.offer_price = 250,
    s.update_time = ?,
```

```
s.version = s.version + 1
WHERE s.id = ?

-- Batch 2: name='Apple', offerPrice=180, id=2
-- (same SQL structure, different parameter values)
```

5.1.5. DELETE Statements

Single Table DELETE

`SQLs.singleDelete()` enters the DELETE chain. Internally it calls `StandardDeletes#singleDelete`.

```
final Delete stmt;
stmt = SQLs.singleDelete()
    .deleteFrom(Stock_.T, AS, "s")    // SingleTableMeta – requires alias
    .where(Stock_.status.equal(StockStatus.DELISTED))
    .asDelete();
```

Generated SQL (PostgreSQL)

```
DELETE FROM stock AS s
WHERE s.status = ? -- param: 'DELISTED'
```

Domain DELETE

`SQLs.domainDelete()` supports `TableMeta` — no `AS` keyword or alias required. Internally it calls `StandardDeletes#domainDelete`, which implements `StandardDelete._DomainDeleteClause`:

```
final Delete stmt;
stmt = SQLs.domainDelete()
    .deleteFrom(Stock_.T, AS, "s")    // TableMeta – any table type
    .where(Stock_.visible.equal(false))
    .asDelete();
```

Generated SQL (PostgreSQL)

```
DELETE FROM stock AS s
WHERE s.visible = ? -- param: false (0)
```

All base interfaces for DELETE are defined in `DeleteStatement`:

Clause	Interface
Single-table	<code>_SingleDeleteFromClause</code> / <code>_SingleDeleteClause</code>
Parent table	<code>_DeleteParentClause</code>
Child table	<code>_DeleteChildClause</code>

5.1.6. Standard Functions

Army provides standard SQL functions through the `Functions` utility class (`impl/Functions.java`). All standard function interfaces are declared in `standard/SQLFunction.java`.

Functions can be used anywhere an `Expression` is expected — in `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, and `HAVING`.

Aggregate Functions

Function	Signature
<code>COUNT(*)</code>	<code>Functions.countAsterisk()</code>
<code>COUNT(expr)</code>	<code>Functions.count(expr)</code>
<code>COUNT(DISTINCT expr)</code>	<code>Functions.count(SQLs.DISTINCT, expr)</code>
<code>SUM(expr)</code>	<code>Functions.sum(expr)</code> , <code>Functions.sum(SQLs.DISTINCT, expr)</code>
<code>AVG(expr)</code>	<code>Functions.avg(expr)</code> , <code>Functions.avg(SQLs.DISTINCT, expr)</code>
<code>MIN(expr)</code>	<code>Functions.min(expr)</code>
<code>MAX(expr)</code>	<code>Functions.max(expr)</code>

```
// Aggregate in SELECT
stmt = SQLs.query()
    .select(Stock_.exchange, Functions.countAsterisk())
    .from(Stock_.T, AS, "s")
    .groupBy(Stock_.exchange)
    .asQuery();

// Aggregate in HAVING
stmt = SQLs.query()
    .select(Stock_.exchange, Functions.countAsterisk())
    .from(Stock_.T, AS, "s")
    .groupBy(Stock_.exchange)
    .having(Functions.countAsterisk().greater(SQLs::literalValue, 100))
    .asQuery();
```

Generated SQL (PostgreSQL)

```
-- Aggregate in SELECT
SELECT s.exchange, COUNT(*)
FROM stock AS s
GROUP BY s.exchange

-- Aggregate in HAVING
SELECT s.exchange, COUNT(*)
FROM stock AS s
GROUP BY s.exchange
HAVING COUNT(*) > 100
```

Scalar Functions

Army provides the full range of standard SQL scalar functions:

Math functions — `acos`, `asin`, `atan`, `atan2`, `ceil`, `cos`, `floor`, `ln`, `log`, `mod`, `power`, `round`, `sign`, `sqrt`, `truncate`, etc.

String functions — `concat`, `length`, `lower`, `upper`, `trim`, `substring`, `replace`, `reverse`, `position`, etc.

Datetime functions — `currentDate`, `currentTime`, `currentTimestamp`, `localTime`, `localTimestamp`, `extract`, `dateAdd`, `dateSub`, `dateDiff`, `datediff`, `timestampAdd`, `timestampDiff`, etc.

Conditional functions — `coalesce`, `nullif`, `greatest`, `least`, `ifNull`, `if`

CASE expression — `Functions.cases()` / `Functions.cases(expr) + .when(...).then(...).elseValue(...).end(type)`

```
// Math: ROUND(offer_price, 2) > 100
Stock_.offerPrice.greater(Functions.round(Stock_.offerPrice, SQLs::literalValue, 2))

// String: CONCAT(exchange, '-', code)
Functions.concat(Stock_.exchange, SQLs::literalValue, "-", Stock_.code)

// Datetime: EXTRACT(YEAR FROM create_time)
Functions.extract(SQLs.WordYear.YEAR, Stock_.createTime)

// CASE: price category
Functions.cases()
    .when(Stock_.offerPrice.greater(SQLs::literalValue,
1000)).then(SQLs::literalValue, "HIGH")
    .when(Stock_.offerPrice.greater(SQLs::literalValue,
100)).then(SQLs::literalValue, "MID")
    .elseValue(SQLs::literalValue, "LOW")
    .end(StandardInfer.STRING)
```

For the complete function reference, see `Functions.java`.

5.1.7. Standard Window Functions

Window functions are provided through the `Windows` utility class (`impl/Windows.java`), implementing `dialect/Window`.

Unlike aggregate functions, window functions require an `OVER` clause which specifies partitioning, ordering, and framing.

Function	Signature	Return Type
<code>ROW_NUMBER()</code>	<code>Windows.rowNumber()</code>	<code>LongType</code>
<code>RANK()</code>	(use dialect API)	—
<code>DENSE_RANK()</code>	<code>Windows.denseRank()</code>	<code>LongType</code>

Function	Signature	Return Type
NTILE(n)	Windows.ntile(n)	LongType
CUME_DIST()	Windows.cumeDist()	DoubleType
LAG(expr)	Windows.lag(expr)	Same as <code>expr</code>
LAG(expr, n)	Windows.lag(expr, n)	Same as <code>expr</code>
LAG(expr, n, default)	Windows.lag(expr, n, default)	Same as <code>expr</code>
LEAD(expr)	Windows.lead(expr)	Same as <code>expr</code>
LEAD(expr, n)	Windows.lead(expr, n)	Same as <code>expr</code>
LEAD(expr, n, default)	Windows.lead(expr, n, default)	Same as <code>expr</code>
FIRST_VALUE(expr)	Windows.firstValue(expr)	Same as <code>expr</code>
LAST_VALUE(expr)	Windows.lastValue(expr)	Same as <code>expr</code>
NTH_VALUE(expr, n)	Windows.nthValue(expr, n)	Same as <code>expr</code>

Each window function returns a `Windows._OverSpec` (or `Windows._WindowAggSpec` for aggregate windows), which exposes the `over()` methods defined by `Window._OverWindowClause`:

```
// Simple OVER()
Windows.rowNumber().over()

// OVER with named window definition
Windows.rowNumber().over("w")

// OVER with inline PARTITION BY + ORDER BY
Windows.rowNumber().over(w -> w
    .partitionBy(Stock_.exchange)
    .orderBy(Stock_.offerPrice.desc())
)

// LAG with offset and default
Windows.lag(Stock_.offerPrice, SQLs::literalValue, 1, SQLs::literalValue, 0)
    .over(w -> w.partitionBy(Stock_.exchange).orderBy(Stock_.createTime.asc()))

// Aggregate window: SUM over a sliding frame
Functions.sum(Stock_.offerPrice)
    .over(w -> w
        .partitionBy(Stock_.exchange)
        .orderBy(Stock_.createTime.asc())
        .rowsBetween(
            Window._FrameUnit.UNBOUNDED_PRECEDING,
            Window._FrameUnit.CURRENT_ROW
        )
    )
```



Aggregate functions as window functions: `Functions.sum(expr)`, `Functions.avg(expr)`, etc. can also be used with `.over(...)` — they become window aggregates. When used without `.over()`, they remain standard aggregates (require `GROUP BY`).

5.1.8. Conditional Expressions

- Equality: `field.equal(...)`
- Inequality: `field.notEqual(...)`
- Greater / Less than: `field.greater(...)` / `field.less(...)`
- Between: `field.between(...)`
- Inclusion: `field.in(...)`
- Null check: `field.isNull()` / `field.isNotNull()`

5.1.9. Logical Operators

- **AND** — Logical conjunction
- **OR** — Logical disjunction
- **NOT** — Logical negation

5.1.10. Default Parameter Binding

When you write predicates like `field.equal(value)`, Army automatically wraps the right-hand value into a parameterized SQL expression. Here's how it works under the hood.

The default path: `field.equal(value)`

Take this simple predicate as an example:

```
Stock_.status.equal(StockStatus.NORMAL)
```

What happens inside Army when this line executes?

1. Step 1 — Call `equal(Object)`

`Stock_.status` is an `OperationTypedField` (extends `OperationTypedExpression`, which extends `OperationExpression`). It calls `OperationExpression#equal(Object)`:

```
return Expressions.biPredicate(this, DualBooleanOperator.EQUAL, operand);
```

2. Step 2 — `biPredicate` → `wrapRight`

[Expressions#biPredicate](https://github.com/ArmyCore/army-core/blob/master/src/main/java/io/army/criteria/impl/Expressions.java#L100) calls [Expressions#wrapRight](https://github.com/ArmyCore/army-core/blob/master/src/main/java/io/army/criteria/impl/Expressions.java#L100)

```
core/src/main/java/io/army/criteria/impl/Expressions.java"
class="extern"><code>Expressions#wrapRight(left</a> to decide how to wrap the right operand
<code>StockStatus.NORMAL</code>.
```

3. Step 3 — `wrapRight` checks `left` type

Because `left` (i.e. `Stock_.status`) implements `TypedExpression`, `wrapRight` enters:

```
else if (left instanceof TypedExpression) {
    rightExp = wrapValue((TypedExpression) left, right);
}
```

It passes the left operand's `MappingType` to `wrapValue`.

4. Step 4 — `wrapValue` checks `LiteralMode`

By default (`LiteralMode.DEFAULT`), it creates a parameterized value:

```
case DEFAULT:
    valueExp = SQLs.param(infer, value); // → JDBC ?
```

5. Step 5 — Result

The generated SQL looks like:

```
status = ? -- with bound value: StockStatus.NORMAL
```



In the default path, Army **never writes the value directly into the SQL string** — it always uses JDBC `?` parameters. This is the safe, recommended default for nearly all application code.

What happens with different value types

`wrapRight` has more branches. Let's see what happens when you pass different kinds of right operands.

Right operand is already an `Expression`

```
// Passing a field reference – already an Expression, used as-is
Stock_.offerPrice.greater(Stock_.price)
// SQL: offer_price > price

// Passing a subquery – already an Expression, used as-is
Stock_.id.in(subQueryExpr)
// SQL: id IN (SELECT ...)
```

When the right operand is already an **Expression** instance (e.g. another field, a subquery, a scalar expression), **wrapRight** simply passes it through without any wrapping.

Left operand has no type information

```
// CTE ref-field – $refField returns a raw SqlField, not a TypedExpression
$refField("normal_stock", Stock_.offerPrice).greater(new BigDecimal("100.00"))
//
//
// This value will be wrapped via
wrapNonNull(right)
//
// Auto-detection: BigDecimal → numeric MappingType
→ SQLs.param()
```

When the left operand is a raw `SqlField` (e.g. from CTE references via `$refField()`), `wrapRight` falls through to `wrapNonNull(right)`, which auto-detects the `MappingType` from `right.getClass()`:

```
// Expressions#wrapNonNull
static ArmyExpression wrapNonNull(final Object value) {
    final MappingType type;
    type = _MappingFactory.getDefaultIfMatch(value.getClass());
    // BigDecimal → NumericMappingType
    // String      → StringType
    // Integer     → IntegerType
    // ...
    return wrapValue(type, value);
}
```

This is why `$refField(...).greater(new BigDecimal("100.00"))` still works — Army matches the Java class to a built-in `MappingType`.

Right operand could trigger an error

```
//  null – wrapRight throws NullPointerException
Stock_.status.equal(null) // right operand must be non-null

//  SQLs.ABSENT – wrapRight throws NPE
Stock_.offerPrice.greater(SQLs.ABSENT) // right operand couldn't be absent
```

Controlling the binding mode: LiteralMode

Once `wrapValue` is called, a **global `LiteralMode`** configuration (property key `expression.literal.mode`) determines whether the value is rendered as `?` or written directly into the SQL string.

The four modes and their effect:

Mode	Generated SQL	When to use
DEFAULT (default)	<code>status = ?</code> ,br (the actual value is sent separately as a JDBC parameter)	Always safe. Recommended for 99% of predicates. Prevents SQL injection.
LITERAL	<code>listingDate = DATE '1970-01-01'</code> ,br (the value is embedded together with its literal type — e.g. <code>DATE</code> , <code>TIME</code> , <code>TIMESTAMP</code> in standard SQL; PostgreSQL additionally supports <code>::type</code> cast syntax, e.g. <code>0::decimal</code>)	Use when you control the value and want to ensure the database interprets it with the correct type. Avoid for user-provided values — SQL injection risk.
CONST	<code>listingDate = '1970-01-01'</code> ,br (the raw value is embedded without any literal type prefix — same as LITERAL minus the type annotation)	Rarely needed. Use when literal type coercion is not required.
PREFERENCE	Numeric/Boolean → <code>price = 100.00</code> (literal),br String → <code>status = ?</code> (param)	Automatically inlines safe types (numbers, booleans) and parameterizes string types. A middle ground between safety and readability.

Here's how each mode translates a real predicate:

```
// Java code: Stock_.offerPrice.greater(new BigDecimal("100.00"))
//             Stock_.status.equal(StockStatus.NORMAL)

LiteralMode.DEFAULT:
offer_price > ?    -- param: 100.00
status = ?        -- param: NORMAL

LiteralMode.LITERAL:
offer_price > 100.00      -- numeric literal with type
status = 'NORMAL'        -- string literal with type
listingDate = DATE '1970-01-01' -- with DATE literal type prefix

LiteralMode.CONST:
offer_price > 100.00      -- raw value, no type annotation
status = 'NORMAL'        -- raw value, no type annotation
listingDate = '1970-01-01' -- without DATE prefix

LiteralMode.PREFERENCE:
offer_price > 100.00      -- BigDecimal → _ArmyNoInjectionType → literal, safe
status = ?               -- enum/string → param, safe
```



The **LiteralMode** applies **globally** to ALL expressions built in the same JVM — it is read once from `army.properties` at class-load time. You cannot mix modes within a single query.

Explicit binding with BiFunction overloads

If you need to override the default binding for a **specific** predicate — regardless of the global `LiteralMode` — use the two-argument overloads that accept a `BiFunction`.

These overloads are defined in `OperationTypedExpression` and `OperationTypedField`. They **bypass** `wrapRight` **entirely** by letting you specify the wrapping function directly:

```
// ① Named-parameter binding – the BiFunction is called with (leftField, value)
//    Useful when left operand has no type (CTE ref-field).
$refField("normal_stock", Stock_.offerPrice).greaterEqual(SQLs::param, 5)
// SQL: normal_stock.offer_price >= ?    -- param name is derived from field

// ② Force literal for a specific predicate, even if the global mode is DEFAULT.
Stock_.status.equal(SQLs::literal, StockStatus.NORMAL)
// SQL: status = 'NORMAL'    -- embedded in SQL

// ③ Use a custom BiFunction to compute the right operand.
//    For example, wrap a collection for an IN clause.
Stock_.status.in(SQLs::namedParam, Set.of(StockStatus.NORMAL, StockStatus.CLOSED))
// SQL: status IN (?, ?)
```

The `BiFunction` protocol:

Overload context	BiFunction signature	Example
<code>field.equal(func, value)</code>	<code>(TypedExpression, T) → Expression</code>	<code>SQLs::param, SQLs::literal</code>
<code>field.spaceEqual(func)</code>	<code>(TypedField, String) → Expression</code>	<code>SQLs::namedParam, SQLs::namedLiteral</code> — uses <code>TypedField#fieldName()</code> as the named parameter key



When should you care about explicit binding?

In everyday application code with normal domain fields (like `Stock_.status`), you almost never need explicit binding — the `DEFAULT` mode is safe and correct.

Explicit binding becomes useful in edge cases: - CTE ref-fields where the left operand has no type information - Force-literal for infrastructure tables (e.g. `sys_config.code`) where injection is not a concern - Custom `BiFunction` implementations for special parameter conventions



The `space*` prefix on `spaceEqual`, `spaceIn`, etc. avoids method overload conflicts with the base interface at the JVM level — these methods are designed for **batch operations** (e.g. `batchUpdate`) where each row's parameter must carry a unique name. They only accept named parameters or named literals (`SQLs::namedParam`, `SQLs::namedLiteral`) because `TypedField#fieldName()` serves as the named parameter key. In normal query operations, prefer the non-`space` overloads (e.g. `equal(value)`).

5.2. CTE (Common Table Expressions)

```
final Select stmt;
stmt = SQLs.query()
    .with("normal_stock").as(ws -> ws
        .select(Stock_.id, Stock_.name, Stock_.offerPrice)
        .from(Stock_.T, AS, "s")
        .where(Stock_.status.equal(StockStatus.NORMAL))
        .asQuery()
    )
    .space()
    .select(s -> s.space("normal_stock", PERIOD, ASTERISK))
    .from("normal_stock")
    .asQuery();
```

Generated SQL (PostgreSQL)

```
WITH normal_stock AS (
    SELECT s.id, s.name, s.offer_price
    FROM stock AS s
    WHERE s.status = ? -- param: 'NORMAL'
)
SELECT normal_stock.*
FROM normal_stock
```

5.3. Subqueries

5.3.1. Scalar Subquery

```
SQLs.scalarSubQuery()
    .select(Stock_.id)
    .from(Stock_.T, AS, "s")
    .where(Stock_.exchange.equal("NYSE"))
    .and(Stock_.code.equal("AAPL"))
    .asQuery()
```

Generated SQL (PostgreSQL)

```
SELECT s.id
FROM stock AS s
WHERE s.exchange = ?
    AND s.code = ?
-- params: 'NYSE', 'AAPL'
-- asQuery() returns Expression, usable in SELECT / WHERE / HAVING
```



The final `asQuery()` at the end of the `SQLs.scalarSubQuery()` chain returns an `io.army.criteria.Expression` instance, **not** a `SubQuery`. It is used in contexts that require a scalar expression, such as `SELECT`, `WHERE`, and `HAVING` clauses. In contrast, the final `asQuery()` at the end of the `SQLs.subQuery()` chain returns a `SubQuery`, which is used in `FROM` clauses or with `IN` / `EXISTS` predicates.

5.3.2. Table Subquery

```
SQLs.subQuery()
    .select("s", PERIOD, Stock_.T)
    .from(Stock_.T, AS, "s")
    .where(Stock_.status.equal(StockStatus.NORMAL))
    .asQuery()
```

Generated SQL (PostgreSQL)

```
SELECT s.*
FROM stock AS s
WHERE s.status = ? -- param: 'NORMAL'
-- asQuery() returns SubQuery, usable in FROM / IN / EXISTS
```

5.4. MySQL Dialect API

5.4.1. The MySQLs Utility Class

MySQL-specific SQL construction entry point: `MySQLs.singleInsert()`, `MySQLs.query()`, etc.

5.4.2. MySQL-Specific Features

- `INSERT IGNORE`
- `INSERT ON DUPLICATE KEY UPDATE`
- `REPLACE INTO`
- `LOAD DATA INFILE`
- MySQL-specific functions

```
final Insert stmt;
stmt = MySQLs.singleInsert()
    .insertInto(Stock_.T)
    .values(stockList)
    .onDuplicateKeyUpdate()
    .set(Stock_.offerPrice, SQLs::values, Stock_.offerPrice)
    .asInsert();
```

```
INSERT INTO stock (id, create_time, update_time, version, exchange,
                  code, name, status, offer_price, listing_date)
VALUES (?, ?, ?, ?, ?,
        ?, ?, ?, ?, ?),
...
ON DUPLICATE KEY UPDATE
  offer_price = VALUES(offer_price),
  update_time = VALUES(update_time),
  version = version + 1
```

5.5. PostgreSQL Dialect API

5.5.1. The Postgres Utility Class

PostgreSQL-specific SQL construction entry point: `Postgres.singleInsert()`, `Postgres.query()`, etc.

5.5.2. PostgreSQL-Specific Features

- **RETURNING** clause — Return field values directly after INSERT/UPDATE
- **ON CONFLICT** clause — Update or ignore on conflict
- **WITH** clause (CTE) — Complex chained operations
- Array operations — **ANY**, **ALL**, array functions
- JSON/JSONB operations — Field extraction, containment queries
- Composite types — Custom composite type reads and writes

```
final Insert stmt;
stmt = Postgres.singleInsert()
    .insertInto(Stock_.T)
    .values(stockList)
    .onConflict(Stock_.name)
    .doUpdate()
    .set(Stock_.offerPrice, SQLs::excluded, Stock_.offerPrice)
    .returning(Stock_.id, Stock_.name)
    .asInsert();
```

```
INSERT INTO stock (id, create_time, update_time, version, exchange,
                  code, name, status, offer_price, listing_date)
VALUES (?, ?, ?, ?, ?,
        ?, ?, ?, ?, ?),
...
ON CONFLICT (name)
DO UPDATE SET
```

```
offer_price = EXCLUDED.offer_price,  
update_time = ?,  
version = version + 1  
RETURNING id, name
```

6. Session API — Executing SQL

The Session API is Army's data operation execution layer, providing a unified entry point for blocking operations.

6.1. SessionFactory Configuration

6.1.1. Configuration Options

- **name** — SessionFactory name
- **packagesToScan** — List of packages to scan for domain classes
- **datasource** — Data source (blocking mode)
- **environment** — Environment configuration (database type, dialect version, etc.)
- **fieldGeneratorFactory** — Field generator factory
- **schemaMeta** — Schema metadata

6.2. Session Operations

6.2.1. Obtaining a Session

Local Session (recommended for short-lived operations)

```
try (SyncLocalSession session = sessionFactory.localSession()) {  
    // Automatic transaction and resource management  
}
```

Regular Session

```
SyncSession session = sessionFactory.session();  
try {  
    // Use the session  
} finally {  
    session.close();  
}
```

6.2.2. Query Operations

```
// Query a list of objects
List<Stock> stocks = session.query(stmt, Stock.class);

// Query a single object
Stock stock = session.queryOne(stmt, Stock.class);

// Query with custom mapping
session.queryObject(stmt, RowMaps::hashMap)
    .forEach(map -> LOG.debug("{} ", map));
```

6.2.3. Update Operations

```
// Insert
long rows = session.update(insertStmt);

// Update
long rows = session.update(updateStmt);

// Delete
long rows = session.update(deleteStmt);
```

6.2.4. Batch Operations

```
long[] rows = session.batchUpdate(batchUpdateStmt);
```

6.3. Transaction Management

6.3.1. Blocking-Mode Transactions

```
try (SyncLocalSession session = sessionFactory.localSession()) {
    session.startTransaction();
    try {
        session.update(stmt);
        session.commit();
    } catch (Exception e) {
        session.rollback();
        throw e;
    }
}
```

6.3.2. Savepoints

```
session.startTransaction();
try {
    session.update(stmt1);
    session.setSavepoint("sp1");
    session.update(stmt2);
    session.commit();
} catch (Exception e) {
    session.rollbackToSavepoint("sp1");
    session.update(stmt3);
    session.commit();
}
```

6.3.3. Transaction Isolation Levels

- `READ_UNCOMMITTED` / `READ_COMMITTED` / `REPEATABLE_READ` / `SERIALIZABLE`

7. Core Architecture

Understanding Army's internal architecture helps with advanced customization and troubleshooting.

7.1. Module Breakdown

The Army framework adopts a modular design with the following core modules:

7.1.1. **army-core (Core Module)**

Standard Criteria API implementation, SQL parsing and generation, metadata management, session management, transaction management, type mapping, and executor support.

7.1.2. **army-annotation (Annotation Module)**

Provides all domain object mapping annotations (`@Table`, `@Column`, `@Generator`, etc.) and the `ArmyMetaModelDomainProcessor` annotation processor for compile-time metamodel class generation.

7.1.3. **army-mysql (MySQL Dialect Module)**

MySQL-specific Criteria API extensions, dialect parser, and DDL parser.

7.1.4. **army-postgre (PostgreSQL Dialect Module)**

PostgreSQL-specific Criteria API extensions, dialect parser, and composite/array type support.

7.1.5. army-oracle (Oracle Dialect Module)

(To be completed)

7.1.6. army-sqlite (SQLite Dialect Module)

SQLite dialect support.

7.1.7. army-jdbc (JDBC Executor Module)

JDBC-based executor implementations ([MySQLExecutor](#), [PostgreExecutor](#), [SQLiteExecutor](#)).

7.1.8. army-spring (Spring Integration Module)

[SessionFactory](#) bean support and Spring transaction manager integration.

7.1.9. army-sync (Blocking Mode Module)

Blocking synchronous API implementation.

7.1.10. army-struct (Foundation Structures Module)

Foundation data structures (such as [CodeEnum](#)) and language utilities.

7.1.11. army-spring-ai-model-chat-memory / army-spring-ai-vector-store

Army implementations of Spring AI chat memory and vector store — bringing database capabilities into the AI ecosystem.

7.2. Architectural Layers

Presentation Layer (Criteria API)

- Generates Statement tree

Parsing Layer (Dialect Parser)

- Dialect-specific SQL + parameter binding

Metadata Layer (Meta)

- Table / field / index metadata queries

Execution Layer (Executor)

- JDBC driver

Session Layer (Session)

- Transaction context, lifecycle management

Type Mapping Layer (Mapping)

- Java ↔ SQL bidirectional type conversion

7.2.1. Presentation Layer (Criteria API)

[SQLs](#) (standard), [MySQLs](#) (MySQL dialect), [Postgres](#) (PostgreSQL dialect) — builds standardized Statement trees using a type-safe DSL.

7.2.2. Parsing Layer (Dialect Parser)

Dialect parsers ([ArmyParser](#) / [MySQLParser](#) / [PostgreParser](#)) convert Statement trees into native SQL for each database and bind parameters.

7.2.3. Metadata Layer (Meta)

- [TableMeta](#) — Table metadata (table name, schema, field chain, primary key fields, discriminator fields)
- [FieldMeta](#) — Field metadata (Java type, mapping type, generator metadata)
- [PrimaryFieldMeta](#) — Primary key field metadata
- [IndexMeta](#) — Index metadata (index name, field list, uniqueness)
- [ServerMeta](#) — Server metadata (database type, version, dialect information)
- [SchemaMeta](#) — Schema metadata registry

7.2.4. Execution Layer (Executor)

The executor interface provides JDBC-based result set processing and batch operations.

7.2.5. Session Layer (Session)

The [Session](#) / [SessionFactory](#) interfaces manage transactions, session context, and lifecycle.

7.2.6. Type Mapping Layer (Mapping)

The type mapping handler is responsible for bidirectional Java ↔ SQL type conversion, supporting both built-in types and custom extensions.

8. Type Mapping System

8.1. Built-in Type Mappings

8.1.1. Numeric Types

- [Byte](#) / [byte](#) · [Short](#) / [short](#) · [Integer](#) / [int](#) · [Long](#) / [long](#)
- [Float](#) / [float](#) · [Double](#) / [double](#)
- [BigInteger](#) · [BigDecimal](#)

8.1.2. String Types

- [String](#) · [Character](#) / [char](#)

8.1.3. Date-Time Types

- `LocalDate` · `LocalTime` · `LocalDateTime`
- `OffsetDateTime` · `ZonedDateTime` · `Instant`
- `Year` · `YearMonth` · `MonthDay`

8.1.4. Other Types

- `Boolean` / `boolean` · `UUID` · `BitSet` · `ZoneId`

8.2. Array Types

- PostgreSQL native array type support: `String[]`, `Integer[]`, `Long[]`, etc.

8.3. JSON Types

- `ArmyJsonType` — JSON type base class, supporting generic mapping (List / Set / Map)

8.4. Custom Type Mappings

8.4.1. Implementing the MappingType Interface

```
public class MyCustomType implements MappingType {
    @Override
    public Class<?> javaType() { return MyCustomClass.class; }

    @Override
    public Object beforeBind(DataType dataType, MappingEnv env, Object source) {
        // Convert before writing to the database
    }

    @Override
    public Object afterGet(DataType dataType, MappingEnv env, Object source) {
        // Convert after reading from the database
    }
}
```

8.4.2. Registering Custom Types

Register custom type mappings through `TypeMappingHandler`.

8.5. JSON / XML Codecs

- `DefaultJacksonCodec` — Jackson-based JSON codec
- `DefaultGsonCodec` — Gson-based JSON codec

- XML codec — To be implemented

9. Field Generators

9.1. Snowflake Generator

A distributed ID generator based on the Snowflake algorithm.

9.1.1. Configuration Parameters

- `startTime` — Start timestamp
- `workerId` — Worker node ID
- `datacenterId` — Data center ID

```
@Generator(type = GeneratorType.PRECEDE,
    value = "io.army.generator.snowflake.Snowflake8Generator",
    params = {
        @Param(name = "startTime", value = "1580224449498")
    })
@Column
private Long id;
```

9.2. UUID Generator

A UUID-based ID generator.

9.3. Custom Generators

9.3.1. Implementing the FieldGenerator Interface

```
public class MyCustomGenerator implements FieldGenerator {
    @Override
    public Object next(FieldMeta<?> field, ReadWrapper domain) {
        // Generation logic
    }
}
```

9.3.2. Registering a Generator Factory

```
public class MyFieldGeneratorFactory implements FieldGeneratorFactory {
    @Override
    public FieldGenerator get(FieldMeta<?> field) {
        GeneratorMeta meta = field.generator();
    }
}
```

```

        if (meta.javaType() == MyCustomGenerator.class) {
            return MyCustomGenerator.create(field);
        }
        throw new IllegalArgumentException("Unsupported generator");
    }
}

```

10. Spring Integration

10.1. SessionFactory Bean Configuration

10.1.1. Blocking Mode

```

@Bean
public ArmySyncSessionFactoryBean sessionFactory(DataSource dataSource) {
    ArmySyncSessionFactoryBean bean = new ArmySyncSessionFactoryBean();
    bean.setDataSource(dataSource);
    bean.setPackagesToScan(Arrays.asList("io.army.example.domain"));
    bean.setFactoryName("army-session-factory");
    return bean;
}

```

10.2. Spring Transaction Management

10.2.1. Blocking Transaction Manager

```

@Bean
public PlatformTransactionManager transactionManager(ArmySyncSessionFactoryBean
sessionFactory) {
    return new ArmySyncLocalTransactionManager(sessionFactory);
}

```

10.3. Declarative Transactions

```

@Transactional
public void createUser(User user) {
    session.update(insertStmt);
}

```

10.4. DataSource Integration

Supports HikariCP, Druid, and all standard **DataSource** implementations.

11. Database Dialect Support Matrix

11.1. MySQL

- **Supported versions:** 5.7, 8.0
- **Blocking:** Supported
- **Special features:** `INSERT ON DUPLICATE KEY UPDATE`, `REPLACE INTO`, `LOAD DATA INFILE`, MySQL-specific functions, JSON type

11.2. PostgreSQL

- **Supported versions:** 12+
- **Blocking:** Supported
- **Special features:** `RETURNING`, `ON CONFLICT`, CTE, array types, JSON/JSONB, composite types, window functions, HNSW vector indexes

11.3. SQLite

- **Blocking:** Supported

11.4. Oracle

- To be completed

12. Advanced Features

12.1. Reserved Field Mechanism

Army reserves the following field names with framework-managed behavior:

Field	Type	Behavior
<code>id</code>	Numeric	Primary key; automatic NOT NULL; non-updatable
<code>createTime</code>	Temporal	Auto-populated on creation; non-updatable
<code>updateTime</code>	Temporal	Auto-populated on update
<code>version</code>	Integer	Optimistic lock version; auto-incremented on update
<code>visible</code>	Boolean	Soft-delete flag; auto-filtered

12.2. Soft Deletes

Use the `visible` field to implement logical deletion. Rows marked as invisible are automatically

filtered out in queries.

```
@Column
private Boolean visible;
```

Visibility mode configuration: **SUPPORT** (enable filtering), **NEVER** (no filtering), **WHITE_LIST** (whitelist mode).

12.3. Optimistic Locking

Use the **version** field for optimistic concurrency control: UPDATE statements automatically validate the version in the WHERE clause; the update fails if the version does not match.

```
@Column
private Integer version;
```

12.4. Inheritance Mapping in Depth

Army uses the single-table inheritance strategy: the parent and all child classes share one database table, with the discriminator column (**@Inheritance**) distinguishing which subtype each row belongs to.

Child classes use **@DiscriminatorValue** to specify their own enum value. The framework automatically filters by the discriminator value in queries.

12.5. Parameter Binding Modes

Mode	Syntax	SQL Output
param	SQLs::param	? parameter placeholder (prepared statement)
literal	SQLs::literal	Inline literal value
namedParam	SQLs::namedParam	Named parameter binding

12.6. SQL Logging

```
envMap.put(ArmyKey.SQL_LOG_MODE.name, SqlLogMode.INFO);
envMap.put(ArmyKey.SQL_LOG_DYNAMIC.name, true);
envMap.put(ArmyKey.SQL_LOG_EXECUTION_TAKE_TIME.name, true);
```

- **SQL_LOG_MODE** — OFF / DEBUG / INFO
- **SQL_LOG_DYNAMIC** — Dynamic logging toggle
- **SQL_LOG_EXECUTION_TAKE_TIME** — Execution-time logging

12.7. Batch Operation Optimization

- Batch insert / batch update / batch delete
- Multi-statement batching

12.8. Result Set Mapping (RowMaps)

```
session.queryObject(stmt, RowMaps::hashMap)
    .forEach(map -> {
        Long id = map.getLong("id");
        String name = map.getString("name");
    });
```

13. Schema Comparison & Synchronization

13.1. SchemaComparer

The Schema Comparer compares the actual database schema against the Army metadata model to detect differences.

13.2. Comparison Process

1. Retrieve actual schema information from the database
2. Compare against the Army metadata model
3. Generate a diff report
4. Generate idempotent DDL statements

13.3. Supported Comparison Items

- Table structure, columns (type / length / precision / nullability), indexes, constraints

13.4. PostgreSQL Composite Type Synchronization

Automatically detect and synchronize composite type definitions at startup, generating idempotent DDL (safe to execute repeatedly without errors).

14. Best Practices

14.1. Domain Class Design

1. Use generics to support inheritance mapping

2. Add comments to all fields
3. Use reserved fields judiciously (`id`, `createTime`, `updateTime`, `version`, `visible`)
4. Configure generators correctly (`PRECED` vs `POST`)
5. Externalize environment-specific configuration through `TableMeta.properties`

14.2. Criteria API Usage

1. Prefer the standard Criteria API (`SQLs`)
2. Use dialect APIs (`MySQLs` / `Postgres`) only when database-specific features are needed
3. Choose parameter binding (`param`) vs literal binding (`literal`) wisely
4. Be aware of the performance implications of CTEs and subqueries

14.3. Transaction Management

1. Use declarative transactions (`@Transactional`)
2. Set appropriate transaction isolation levels
3. Be mindful of the performance impact of long-running transactions
4. Use savepoints for fine-grained control

14.4. Performance Optimization

1. Use batch operations instead of looping single-row operations
2. Use indexes wisely (create them during DDL generation via `@Index`)
3. Avoid N+1 query problems
4. Use appropriate parameter binding strategies

14.5. Type Mapping

1. Prefer built-in type mappings
2. Use custom type mappings with caution
3. Configure JSON/XML codecs correctly
4. Pay attention to timezone handling

15. FAQ

15.1. Metamodel Classes Not Generated by the Annotation Processor

Check: 1. Whether `annotationProcessors` is configured correctly 2. Whether the domain class uses

the `@Table` annotation 3. Whether the compile classpath includes the `army-annotation` dependency

15.2. Snowflake Generator Duplicate startTime Warning

Issue: Using the same `startTime` parameter across multiple domain classes risks ID collisions.

Solution: 1. Use distinct `startTime` values for different entities 2. Suppress the warning with `-Army.snowflakeStartTimeWarning=false`

15.3. Transaction Exception (IllegalTransactionStateException)

Solution: 1. Verify that a transaction has been properly started 2. Check the transaction propagation behavior configuration 3. Ensure database operations are executed within a transaction context

15.4. Type Mapping Errors

Solution: 1. Check whether the field type matches the database type 2. Configure the correct `MappingType` 3. Verify the JSON/XML codec configuration

16. Appendix

16.1. Reserved Fields Reference

- `id` — Primary key
- `createTime` — Creation timestamp
- `updateTime` — Update timestamp
- `version` — Optimistic lock version number
- `visible` — Soft-delete visibility flag

16.2. Database Support Matrix

Database	Version	Blocking	Special Features
MySQL	5.7, 8.0	☐	INSERT ON DUPLICATE KEY UPDATE, LOAD DATA
PostgreSQL	12+	☐	RETURNING, ON CONFLICT, array types, composite types, HNSW
SQLite	—	☐	—
Oracle	—	Planned	—

16.3. Version History

16.3.1. 0.6.8-SNAPSHOT (Current)

- Under active development

16.3.2. 0.6.6

- Stable release
- Improved MySQL and PostgreSQL support

16.3.3. 0.6.0

- Major architecture overhaul
- Introduced the new Criteria API

16.4. Related Resources

- GitHub: <https://github.com/PillArmy/army>
- Documentation: <https://army.qinarmy.io>
- Organization: <https://qinarmy.io>

16.5. License

Apache License 2.0